

Vibecoder 技术名词入门词典

用生活比喻讲清 122 个高频技术名词：它有什么用、什么时候会遇到，以及新手最常踩的坑。

[← 返回技能工具](#)

122 个词条 7 大分类 每词 4 个维度 + 关联跳转 更新于 2026-06-05



01 上网与软件基础

每天都在用、却说不清的最底层概念

13 词

服务器 Server

服务器就是一台托管在机房、24 小时开机的计算机,你电脑关了它还在跑,所以网站才能随时随地被人访问。它就像一家从不打烊的便利店,谁来都能进。

价 值 它是网站和应用真正运行的地方。你的代码、数据库、图片都放在服务器上,全世界用户通过网络向它请求内容。没有服务器,程序只能在你自己电脑上跑,别人用不了。

何 时 用 到 vibecoder 想让做好的网站或小程序后端被别人访问,就得把代码放到服务器上(自己租云服务器,或用 Vercel、Railway 等托管平台代管)。接口、数据库通常都跑在服务器上。

常 见 坑 新手以为服务器越贵越好,其实小项目一台几十块的入门云主机就够。另一个坑:服务器一直开机就一直计费,测试项目用完忘了关,账单会悄悄累积。

客户端

后端

IP地址

带宽/流量

客户端 Client

客户端就是你这一端发出请求的设备或软件,比如你手机上的 App、电脑上的浏览器。它像餐厅里点菜的客人,服务器则是后厨,客人下单、后厨做好端上来。

价 值 理解客户端/服务器这对关系,是理解整个互联网怎么运转的基础。你点一下、刷新一下,都是客户端在向服务器要东西,服务器把结果送回来显示。

何 时 用 到 vibecoder 看 AI 生成的代码时,会看到代码分两部分:跑在用户设备上的(客户端)和跑在服务器上的(服务端)。搞清一段代码属于哪端,才知道数据从哪来、密钥能不能放这。

常 见 坑 最危险的坑:把 API 密钥、数据库密码写在客户端代码里。客户端代码用户能直接看到,密钥等于公开。敏感操作和密钥必须放在服务器端。

服务器

前端

后端

前端 Frontend

前端就是网站给你看的那一面——页面长什么样、按钮放哪、点了有什么动画。好比一家店的门面装修、招牌、货架摆设,顾客直接看到、摸到的全部。

价值 前端决定用户的第一印象和使用体验。同样的功能,前端做得好用顺手,用户才留得住。它用 HTML、CSS、JavaScript 这些技术,跑在用户的浏览器里。

何时用到 vibecoder 让 AI '做一个登录页面'把按钮改成蓝色'加个加载动画',改的都是前端。做落地页、作品集、工具界面,主要工作量都在前端。

常见坑 前端只负责显示,别把它当保险箱。比如'只在前端校验密码'前端隐藏了付费按钮'都拦不住懂行的人,真正的权限和验证必须要在后端再做一遍。

后端

客户端

后端 Backend

后端是网站看不见的那一面——后厨。你在前台点了'登录',后端去核对账号密码、查数据库、算结果,再把答案递回前台。顾客看不到后厨,但菜全在那做。

价值 后端处理真正的逻辑和数据:验证身份、读写数据库、扣款、调用第三方服务。它跑在服务器上,负责前端做不了、也不能信任前端去做的事(比如算钱、改数据)。

何时用到 vibecoder 做'用户注册并保存到数据库'生成订单并扣款'调 AI 接口返回结果',这些都是后端活。一旦项目要存数据、要登录、要付费,就绕不开后端。

常见坑 新手常把'前端看起来能用'当成'功能完成'。比如注册页填完看似成功,但没写后端把数据存进数据库,刷新就没了。前端是壳,后端才真正干活。

前端

服务器

数据库

API 应用程序接口

网址 URL

URL 就是一个网页的完整门牌地址,精确到具体哪一页。好比'北京市朝阳区xx路5号3单元201室'——不只是哪栋楼(网站),而是楼里的哪一个房间(具体页面)。

价 值 URL 让你能直接跳到互联网上任意一个具体页面或资源。复制粘贴一个 URL 给别人,他就能看到和你一模一样的内容。它是浏览器、链接、分享的基础。

何 时 用 到 vibecoder 部署完网站会拿到一个 URL,发给别人就能访问。调 API 时也要填对方的 URL(接口地址)。做页面跳转、配置路由,本质都是在处理 URL。

常 见 坑 URL 里 http 和 https 一字之差,安全性天壤之别;末尾带不带斜杠、大小写有时也会指向不同页面。复制接口地址时多一个空格、少一个字母都会导致请求失败。

域名

HTTP/HTTPS

端口

域名 Domain Name

域名就是网站好记的名字,比如 taobao.com。互联网内部其实是用一串数字(IP 地址)找电脑的,但数字记不住,域名就像把'13800138000'存成'老王'一样,给冰冷的地址起个人能记的名。

价 值 域名让你的网站有一个专业、好记、可信的入口。注册一个域名就把这个名字占为己有,别人输入它就能到你的站。它也是品牌的一部分。

何 时 用 到 vibecoder 网站做好要正式上线时,去阿里云/腾讯云/Cloudflare 等注册一个域名(.com 国内大约 75-95 元/年),绑到部署平台上,网址就从一串随机字符变成你自己的品牌名。

常 见 坑 域名是按年租的,不是买断,忘记续费会被别人抢注,站就没了。注册时也别只图首年 1 元的特价,看清第二年续费价(常涨到几十上百)。

DNS 域名解析

IP地址

网址

IP地址 IP Address

IP 地址是每台联网设备在网络里的门牌号,一串数字如 142.250.x.x。就像每部手机有唯一号码,数据要送到哪台电脑,靠的就是认这个号。域名只是这个号好记的别名。

价 值 它是网络通信能精准送达的根本。你访问任何网站,最终都是把请求送到对方服务器的 IP;服务器也靠你的 IP 把内容回送给你。没有 IP,数据不知道发去哪。

何 时 用 到 vibecoder 租了云服务器,平台会给你一个公网 IP,你用它远程连服务器、部署代码。配置域名解析时,本质就是把域名指向这个 IP。排查'访问不了'时也常查 IP 不通。

常 见 坑 IP 直接暴露在外容易被扫描攻击,正式项目常用 Cloudflare 等挡在前面隐藏真实 IP。另外家用宽带的 IP 会变动,不能直接拿来长期对外提供服务。

域名

DNS 域名解析

服务器

端口

DNS 域名解析 DNS

DNS 就是互联网的电话簿。你说要找'taobao.com'这个名字,DNS 帮你查出它对应的数字门牌号(IP),浏览器才知道该去敲哪台电脑的门。每次访问网站都先悄悄查一次这本簿子。

价 值 DNS 让你只记好记的域名,不用记一长串数字。它把人类用的名字翻译成机器用的 IP,是域名能用起来的关键一环。改解析记录,就能把域名指向新的服务器。

何 时 用 到 vibecoder 买完域名要让它指向自己的服务器/部署平台,就在域名后台'添加解析记录'(填 A 记录指向 IP,或 CNAME 指向平台域名),这一步就是配 DNS。

常 见 坑 DNS 解析改完不是马上生效,可能要几分钟到几小时(叫'生效延迟'),期间有人能访问有人不能,别以为配错了反复改。记录类型(A / CNAME)填错也是新手高频坑。

域名

IP地址

服务器

HTTP/HTTPS HTTP/HTTPS

HTTP 是浏览器和服务端之间说话用的'普通话'——规定了怎么问、怎么答。HTTPS 是给这段对话加了把锁的加密版,别人在中途偷听也只看到乱码。差一个 S,等于明信片 and 密封信的区别。

价 值 HTTP 让浏览器和服务端能互相听懂、传数据。HTTPS 在此基础上加密,保护密码、支付信息不被窃取。现在没有 HTTPS 的网站,浏览器会直接标红'不安全'吓退用户。

何 时 用 到 vibecoder 部署网站时,Vercel、Cloudflare 等平台默认自动配好 HTTPS(免费证书),你基本不用管。但接支付、做登录时,必须确认走的是 https,否则数据裸奔。

常 见 坑 别贪省事用 http。一旦页面有登录或支付而走 http,信息全程明文传输,极易被截。另外页面是 https 但里面引用了 http 的图片/脚本,浏览器会报'混合内容'警告。

网址

服务器

API 应用程序接口

端口 Port

如果说 IP 地址是一栋大楼的门牌号,端口就是楼里的具体房间号。同一台服务器(同一个 IP)上可以同时跑很多服务,靠端口区分:网站走 443 房间,数据库走 5432 房间,各收各的客。

价 值 端口让一台电脑能同时提供多种服务而不打架。访问网站默认走 443(HTTPS)或 80(HTTP),所以你平时不用输端口。但跑多个服务时,端口就是把它们分开的关键。

何 时 用 到 vibecoder 本地开发时常看到 localhost:3000、localhost:8080,后面那串数字就是端口。部署时若服务连不上,十有八九是端口没开放或被别的程序占用了。

常 见 坑 本地能跑(因为端口对你自己开着)不代表线上能访问,服务器防火墙默认关大部分端口,要手动放行。另外两个程序抢同一个端口会报'端口已被占用',换个号即可。

IP地址

服务器

网址

HTTP/HTTPS

数据库 Database

数据库就是网站的超级 Excel 表格 + 档案柜,专门有条理地存放用户、订单、文章这些数据,并且能瞬间查到、改对。和你电脑上的文件不同,它能同时被成千上万人安全读写。

价 值 凡是要'记住'东西的功能都靠数据库:用户注册了得存账号,下单了得存订单,发帖了得存内容。它保证数据持久、不丢、查得快,是后端的核心地基。

何时用到 vibecoder 做的项目一旦需要登录、保存、列表展示,就得接数据库。常用 Supabase、PostgreSQL、MySQL 等。AI 写后端代码时,增删改查的对象基本都是数据库。

常见坑 最大的坑是把数据库密码或连接串写进前端代码,等于把档案柜钥匙挂门口。另外删数据前没备份、用字符串拼 SQL 导致被注入攻击,都是常见且致命的错误。

后端

服务器

API 应用程序接口

客户端

API 应用程序接口 API

API 就像餐厅的菜单 + 服务员。你不用进后厨,只要照菜单点单(发请求),服务员就把做好的菜端出来(返回数据)。它是两个程序之间约定好的'怎么点、怎么给'的窗口。

价 值 API 让你不必自己造轮子,直接调用别人做好的能力:调地图 API 显示位置,调支付 API 收钱,调 AI 大模型 API 让程序会聊天。它是把各种服务拼装成你产品的接口。

何时用到 vibecoder 几乎天天用:接 OpenAI/Claude 的 API 让 App 有 AI 能力,接 Stripe 的 API 收款,接天气/地图 API 取数据。前端和自己的后端之间通信,也是通过 API。

常见坑 调 API 几乎都要密钥(API Key),它等于你的付费账号密码,千万别写进前端或传到公开的 GitHub,泄露后别人能盗刷你的额度。另外多数 API 有调用次数和频率限制。

后端

HTTP/HTTPS

数据库

客户端

带宽/流量 Bandwidth

带宽是水管的粗细,流量是流过的水总量。管子越粗(带宽大),同一时间能过的水越多,网站打开越快;水流过的总量(流量)则按月累计,用多了要么变慢要么加钱。

价 值 带宽决定网站快不快、能扛多少人同时访问;流量决定你每月传了多少数据出去。视频、大图片特别耗流量。理解它能帮你看懂云服务账单,避免突然被收一大笔。

何 时 用 到 vibecoder 网站突然很多人访问、或放了大量图片视频时,会撞到带宽/流量上限——表现为变慢,或账单暴涨。选托管平台、配 CDN 时,都要看免费流量额度够不够。

常 见 坑 免费托管平台都有流量额度,放高清视频/大图很容易超额,超了要么降速要么扣费。新手常忽略图片没压缩导致流量翻倍。用 CDN 缓存能大幅省流量、提速。

服务器

IP地址

网址

云 · 部署 · 运维

02

把做好的东西放上网、让全世界访问、保证不挂

22 词

云计算 Cloud Computing

以前用电得自己买台发电机摆家里,又贵又要维护;云计算就是改成接市电——别人建好大电厂,你按用多少电交多少钱,随用随取。计算和存储也一样,不用自己买机器,在网上租别人的。

价 值 你不用花几万块买服务器、不用懂怎么装机维护,做好的网站和应用直接放到云上,几分钟就能让全世界访问,按实际用量付费,用得少就花得少。

何 时 用 到 当你用AI做好一个网站、小程序或后端,想让它真正跑在网上让别人能打开时,第一件事就是选一家云服务商(阿里云、腾讯云、AWS等)开通账号。

常 见 坑 以为云就一定便宜。算力、流量、存储分开计费,流量和数据库尤其容易超支;新手常忘了关闭置资源,月底账单吓一跳。开通前先看清各项单价。

云服务器/

对象存储

Serverless 无服务器

云服务器/VPS Cloud Server / VPS

就是一台托管在机房、24小时开机的电脑,你电脑关了它还在跑,所以网站才能一直访问。你花钱租它的一部分,像在共享办公楼里租一个独立的小隔间,有自己的钥匙和地址。

价 值 这是放网站和后端程序最常用、最灵活的地方。你能完全控制这台机器,想装什么、怎么配都行,价格也便宜,入门款一个月几美元就能起步。

何 时 用 到 AI帮你写完一个后端服务或网站,要找个地方让它一直运行,最常见的就是租一台云服务器,把代码传上去跑起来。

常 见 坑 入门款配置低(常见1核1G内存),稍微复杂的应用一跑就卡甚至崩。另外服务器要自己负责安全更新和防黑客,买了不管会被入侵。新手可先选带一键部署的服务商。

云计算

部署

SSH 远程登录

部署 Deploy

你在自己家厨房做好一道菜(本地写好的代码),部署就是把这道菜的做法和食材搬到餐厅后厨,让餐厅能批量做给客人吃。代码从你电脑搬到服务器、跑起来对外服务,这个搬运+启动的过程就叫部署。

价 值 代码只待在你电脑上,只有你能看;部署之后别人才能通过网址访问。这是从'我做了个东西'到'大家能用我做的东西'的关键一步。

何 时 用 到 AI帮你把功能都写好、本地测试通过后,你想发给朋友看或正式上线,就要做部署——把代码传到服务器并启动。每次改完代码想更新线上版本,也要重新部署。

常 见 坑 本地能跑不代表线上能跑。本地装了的東西服务器上沒有、配置不一样、端口没开,都会导致部署后打不开。第一次部署踩坑最多,别指望一次成功。

云服务器/

上线/生产环境

环境变量

上线/生产环境 Production

生产环境就是正式开门营业的店面,进来的是真客人,出了错真客户会看到、会投诉。和它对应的是后厨试菜的厨房(测试环境)——那里随便搞砸都没事,客人看不到。

价 值 区分'正式对外'和'内部试验'两个地方,你才能放心大胆地在试验区折腾,确认没问题了再放到正式区。避免一改代码就直接影响真实用户。

何 时 用 到 当你的产品已经有真实用户在用,任何改动都要先在测试环境验证,再'上线'(发布到生产环境)。「上线了吗」='正式版更新了没'。

常 见 坑 直接在生产环境改代码、调数据库,是新手大忌——一个手抖全站挂掉,用户数据可能没了。另外别把测试用的假数据、调试开关带到生产环境。

开发/测试/生产环境

部署

监控

开发/测试/生产环境 Environments (Dev / Staging / Production)

像做菜三个阶段:开发=你在家随便试新菜谱;测试=请几个朋友先尝尝挑毛病;生产=正式上菜给付钱的客人。三套各自独立的厨房,互不干扰,坏了不影响下一个。

价 值 有了分层,你可以在开发和测试环境放心试错,把问题挡在用户看不到的地方,只让经过验证的版本进入生产。这是不让真实用户当小白鼠的保障。

何 时 用 到 项目稍微正式一点就需要至少'开发(自己电脑)'和'生产(线上)'两套;有团队或真实用户后再加'测试环境'。每套用各自独立的数据库和配置。

常 见 坑 最常见的坑是三套环境配置不一致——测试时连的是测试数据库,上线却忘了改回生产数据库,结果用户数据写错地方。每套环境的配置要严格分开管理。

上线/生产环境

环境变量

部署

环境变量 Environment Variable

像保险箱的密码,不写在菜谱里(代码里),而是单独贴在厨房墙上一张只有员工能看的纸条上。换个厨房(换台服务器)纸条内容可以不一样,菜谱本身不用改。

价 值 把密码、API密钥这类敏感信息和会随环境变的配置,从代码里抽出来单独存放。好处:代码可以公开分享而不泄露密钥;同一份代码在不同环境只换配置就能跑。

何 时 用 到 接入支付、调用AI接口、连数据库时都会拿到密钥,这些绝不能直接写进代码,而要放进环境变量。换到生产环境时,改环境变量而不动代码。

常 见 坑 头号大坑:把密钥直接写在代码里又上传到GitHub,几分钟就被人扫到盗刷,账单爆炸。一定用环境变量,且把存密钥的文件(如.env)加进.gitignore不上传。

开发/测试/生产环境

SSL/TLS 证书

部署

容器 Container

像一个标准化的集装箱:把你的应用和它需要的所有东西(代码、依赖、配置)一起打包封箱。不管搬到哪条船、哪个港口(哪台服务器),箱子里的东西原样不变,拆开就能用。

价 值 彻底解决'我电脑上能跑、服务器上跑不了'的老大难问题。把运行环境一起打包,换台机器照样跑,部署变得可预测、可复制。

何 时 用 到 当你被'本地正常、线上报错'折磨,或想让应用在任何服务器上一键跑起来时,会把应用装进容器。现代部署流程几乎都基于容器。

常 见 坑 容器是临时的,里面写的文件重启就没了——数据库、上传的图片必须存到容器外面(挂载卷或对象存储),否则一重启数据全丢。新手常栽在这。

Docker

镜像

Kubernetes / K8s

Docker Docker

Docker就是制造和搬运'集装箱'(容器)的那套标准工具和码头设备。它定义了箱子怎么打包、怎么开关、怎么运。说'用Docker'基本等于'用容器这套玩法'。

价 值 它是容器技术里事实上的标准工具,用一个简单的配置文件(Dockerfile)就能把应用打包成标准箱子,几乎所有云平台 and 教程都支持它,生态最成熟。

何 时 用 到 AI帮你部署项目时,常会让你写个Dockerfile、运行docker命令,把应用打包成镜像再跑成容器。租的服务器上装好Docker几乎是部署标配。

常 见 坑 打出来的镜像动辄几个G、构建超慢,是因为没做精简(用了臃肿的基础镜像、把没用的文件也打进去)。另外别把密钥写进Dockerfile,会被打进镜像泄露。

容器

镜像

Kubernetes / K8s

镜像 Image

镜像像一张'系统光盘'或菜谱模板:它是封好的、只读的应用打包成品。容器则是用这张光盘装出来的、正在运行的那台机器。一张镜像可以开出很多个一模一样的容器。

价 值 镜像让应用的打包成品可以保存、分享、版本管理。做好一次,到处复制运行,每个都完全一致。它是容器能'到处一样跑'的底层载体。

何 时 用 到 用Docker打包应用时,产出的就是镜像;部署时把镜像推到镜像仓库,服务器再拉下来跑成容器。AI部署脚本里常出现build镜像、push/pull镜像的步骤。

常 见 坑 镜像有版本标签(tag),新手爱用latest这个标签,结果不同时间拉到的内容不一样,线上出现'昨天好好的今天就崩了'。生产应固定明确的版本号。

Docker

容器

Kubernetes / K8s

Kubernetes / K8s Kubernetes / K8s

如果一个容器是一名工人,K8s就是大型工厂的智能调度主管:自动安排几百名工人上岗、谁累趴了立刻换人顶上、订单暴增就临时加派人手、订单少了就让人休息。它管的是一大群容器。

价 值 当应用大到一台机器扛不住、需要几十上百个容器协同时,K8s自动负责扩缩容、故障自愈、流量分配,让大规模系统不靠人盯着也能稳定运行。

何 时 用 到 个人项目和中小应用基本用不到。等到用户量很大、要多台服务器跑成集群、追求高可用时才会上K8s。名字里的8是'K和s之间8个字母'的缩写。

常 见 坑 最大的坑是过早使用——给个小网站上K8s,复杂度暴涨、运维成本高得离谱,纯属杀鸡用牛刀。小项目一台云服务器或Serverless足够,别被'高大上'忽悠。

容器

Docker

负载均衡

对象存储 Object Storage (S3)

像一个无限大的网盘仓库,专门放用户上传的图片、视频、文件。每个文件存进去都给一个固定网址,谁都能凭网址下载。不像服务器硬盘有容量上限,这个仓库几乎装不满。

价 值 用户头像、商品图、上传的文档这类文件,放服务器硬盘上既占空间又难扩展,放对象存储则便宜、无限扩容、还能直接给出下载链接。AWS S3是最有名的对象存储,价格约每GB每月0.023美元。

何 时 用 到 做带'上传图片/文件'功能的应用时,几乎一定用对象存储来存这些文件,而不是塞进服务器或数据库。配上CDN还能让全球用户下载得飞快。

常 见 坑 权限设置最容易出错:设成公开可读可写,等于把仓库大门敞开,别人能随便上传垃圾甚至看到你的私密文件。要严格区分哪些文件能公开、哪些只能私有访问。

CDN 内容分发网络

云服务器/

环境变量

CDN 内容分发网络 CDN (Content Delivery Network)

像连锁便利店:总仓在北京,如果广州人每次都从北京发货就很慢。CDN在全国各地都开了分店,提前把热门货(图片、视频、网页)铺到离你最近的店,你就近取,秒到。

价 值 让全球用户打开你的网站、加载图片视频都很快,不管他在哪个国家。同时帮你分担流量、抵挡一部分恶意攻击,源服务器压力小很多。

何 时 用 到 网站有较多图片视频、或用户分布在不同地区、或访问量大时,套上CDN能明显加速。图片视频类、出海的应用几乎必用。

常 见 坑 CDN会缓存内容,你更新了文件但用户还看到旧版,是因为缓存没刷新——这就是'清缓存'的由来。改完静态文件记得清CDN缓存,否则改了像没改。

对象存储

反向代理 /

缓存

反向代理 / Nginx Reverse Proxy / Nginx

像酒店前台:客人(访问者)不直接冲进后厨,都先到前台。前台再决定把你领到哪个房间(哪个后端服务)、帮你登记、挡掉捣乱的人。Nginx就是最常用的这种'前台'软件。

价 值 它站在用户和你的后端程序之间,负责分发请求、处理HTTPS加密、挡掉部分攻击、把多个服务统一到一个网址下。让你的后端不用直接暴露在外,更安全也更灵活。

何 时 用 到 几乎所有正式上线的网站都会用Nginx当门面:配置域名、装SSL证书让网址变https、把/api开头的请求转给后端、其他转给前端页面,都靠它。

常 见 坑 配置文件语法严格,一个分号、一个路径写错就整站502/404打不开,且报错信息不直观,新手调起来头大。改完配置务必先测试再重载。

负载均衡

SSL/TLS 证书

云服务器/

负载均衡 Load Balancer

像银行门口的引导员:只开一个窗口会排长队,引导员把客人均匀分到多个窗口,谁空闲去谁那,大家都快。它把海量访问平摊到多台服务器上。

价 值 一台服务器扛不住的访问量,可以加几台一起扛,负载均衡负责公平地把请求分给它们。某台坏了就自动不再往它派活,网站不会因为一台机器挂了就整体瘫痪。

何 时 用 到 用户量增长到一台服务器明显吃力、或对稳定性要求高(不能挂)时,会加多台服务器并在前面放负载均衡。个人小项目一般还用不到。

常 见 坑 多台服务器后,用户登录状态(session)如果只存在某一台上,下次请求被分到另一台就'被登出'了。要把这类状态存到共享的地方(如Redis),否则用户体验诡异。

反向代理 /

Kubernetes / K8s

Redis

SSH 远程登录 SSH

像给远方那台一直开机的服务器配了一把带加密的远程钥匙:坐在自己家就能登进那台机器的命令行,像直接坐在它面前一样敲指令操作。全程加密,别人偷看不到。

价 值 服务器在机房里你摸不到,SSH就是你操作它的主要通道:登上去传代码、装软件、看日志、重启服务。没有它你基本没法管理一台云服务器。

何 时 用 到 租了云服务器后,第一步通常就是用SSH登进去做初始配置;之后每次手动部署、排查线上问题、改配置,都靠SSH连上去操作。

常 见 坑 用简单密码登录的服务器,几小时内就会被全球扫描机器人暴力破解。务必改用密钥登录、关掉密码登录,并妥善保管私钥文件——私钥泄露等于钥匙被偷。

云服务器/

部署

环境变量

SSL/TLS 证书 SSL/TLS Certificate

像给网站发的一张官方身份证+加密信封。有了它,网址前面就从http变成https,浏览器显示小锁,用户和网站之间传的密码、银行卡号被装进加密信封,中间人偷看到的只是乱码。

价 值 保护用户输入的敏感信息不被窃听,同时向用户证明'这网站是真的,不是钓鱼假冒'。没有它,浏览器会标红'不安全',用户不敢用,支付等功能也无法接入。

何 时 用 到 任何正式网站、尤其涉及登录和支付的,都必须装SSL证书让网址变https。微信小程序、支付接口等更是强制要求https,否则根本调不通。

常 见 坑 免费证书(如Let's Encrypt)有效期通常90天,忘了续期到期那天全站突然变'不安全'打不开。一定要配置自动续期;另外http和https不可混用,会报安全警告。

反向代理 /

环境变量

上线/生产环境

Serverless 无服务器 Serverless

不是真的没服务器,而是像打车代替买车:你不用买车养车(租服务器、管运维),要用时一招手车就来(代码被触发就运行),下车按里程付费(按调用次数和时长计费),没人用时一分钱不花。

价 值 省去管理服务器的全部麻烦,不用关心扩容、运维、闲时资源浪费。访问量大了它自动扩,没访问就不计费,特别适合流量不稳定或刚起步的项目,成本可以极低。

何 时 用 到 做个小接口、定时任务、表单提交处理,或不想折腾服务器运维时,用Serverless(如各云的函数计算、Vercel、Cloudflare Workers)几分钟上线,几乎零运维。

常 见 坑 很久没人调用时,第一次访问会有几秒延迟(冷启动)。另外它适合短小任务,不适合长时间运行的活;用量一旦暴涨,按次计费可能反而比包月服务器贵。

云计算

对象存储

部署

日志 Log

像监控录像+店员的值班记录:程序每做一件事、每出一次错,都记一笔,写明几点几分发生了什么。出了问题,翻这些记录就能还原'当时到底发生了啥'。

价 值 线上出问题时你不在现场、也没法暂停复现,日志是唯一能告诉你'刚才哪一步崩了、报了什么错'的依据。排查bug、定位故障全靠它。

何 时 用 到 用户反馈'页面打不开'、'支付失败'时,你登上服务器或打开日志面板,翻当时的报错记录来找原因。AI帮你debug线上问题,第一句往往是'把日志贴出来'。

常 见 坑 两个极端坑:一是啥也不记,出事两眼一抹黑;二是把密码、银行卡号等敏感信息也打进日志,造成泄露。还有日志不清理会把硬盘撑满导致服务挂掉。

监控

上线/生产环境

SSH 远程登录

监控 Monitoring

像给服务器装上心电图和报警器:实时盯着CPU、内存、访问量、有没有报错,一旦指标异常(宕机、错误飙升)就自动给你发消息,让你在用户投诉前就知道出事了。

价 值 日志是出事后翻查,监控是出事前/出事时主动报警。它让你不必整天盯屏幕,系统一有风吹草动就通知你,把故障扼杀在用户感知之前。

何 时 用 到 产品有真实用户、不能随便挂的时候,会配监控+告警(如服务器宕机、接口大量报错、磁盘快满时自动通知微信或邮件)。个人项目至少配个'网站挂了通知我'。

常 见 坑 告警设得太敏感,天天误报,你慢慢就不看了,真出事反而漏掉(狼来了效应)。要把告警阈值调到'报的都是真要管的事',只在真正重要的指标上报警。

日志

上线/生产环境

负载均衡

缓存 Cache

像把常用的东西放在手边抽屉,而不是每次都跑去仓库取。把算过一次、查过一次的结果先存在快的地方,下次有人要直接给,不用重新算、重新查数据库,快得多。

价 值 大幅提升速度、降低数据库压力。同样的数据反复被很多人请求时,缓存一次、复用千百次,网站秒开,服务器也轻松。是性能优化最常用的第一招。

何 时 用 到 首页热门内容、不常变的配置、调用一次很慢的接口结果等,都适合缓存。当你的网站'打开有点慢'数据库扛不住'时,加缓存往往立竿见影。

常 见 坑 经典难题:数据更新了但缓存还是旧的,用户看到过期内容(缓存失效问题)。改了数据要记得让对应缓存过期或更新,否则'明明改了却不生效'会让你抓狂。

Redis

CDN 内容分发网络

负载均衡

Redis Redis

像放在桌上的一块超快的便签板:数据直接记在内存里,读写快到飞起,但断电(重启)默认会擦掉。专门用来放需要频繁、极速读写的临时数据。

价 值 它是做缓存、存登录状态、做排行榜/计数器、简单消息队列的瑞士军刀,因为快、好用、生态成熟而几乎成为标配。多台服务器共享session、防止重复提交也常靠它。

何 时 用 到 给网站加缓存提速、存用户登录态、做限流防刷、做实时排行榜或在线人数统计时,基本都会上Redis。多服务器架构里它几乎是必备组件。

常 见 坑 默认数据在内存里,重启或内存满了可能丢——别拿它当唯一的数据库存重要数据。还有它早期默认无密码且对外开放,被黑客扫到直接入侵,务必设密码、别裸奔。

缓存

消息队列

负载均衡

消息队列 Message Queue

像餐厅的取餐叫号系统:你点完单(下了任务)立刻拿号走人,不用站在窗口干等;后厨按号慢慢做,做好了叫你。把'提交任务'和'实际处理'拆开,谁也不堵着谁。

价 值 把耗时的活(发邮件、生成报告、处理上传视频)丢进队列后台慢慢做,用户点一下就立刻得到响应,不用转圈等待。流量高峰时任务排队处理,不会一下子压垮系统。

何 时 用 到 功能里有'点了之后要等很久'的操作(发短信、调用AI生成、批量处理),或要削峰填谷扛突发流量时,用消息队列(如RabbitMQ、Kafka、Redis队列)把它们异步化。

常 见 坑 消息可能重复送达或处理失败,如果是扣款、发货这类操作,处理逻辑必须做到'同一条消息执行两次结果也一样'(幂等),否则会重复扣款、重复发货。

Redis

缓存

Serverless 无服务器

写代码与协作

03

动手做(或让 AI 做)东西时绕不开的开发概念

25 词

编程语言 Programming Language

就像你和外国人沟通得用一门语言,你想让电脑干活也得用电脑听得懂的语言。中文、英语是给人用的,Python、JavaScript 是给电脑用的,你写出指令,电脑照着做。

价 值 电脑只认 0 和 1,人没法直接用。编程语言是中间的翻译层,让你用接近人话的方式写指令,电脑再翻译成它能执行的动作。不会它,你连让电脑做最简单的事都开不了口。

何时用到 你让 AI 帮你做网站或小程序时,它生成的全是某种编程语言的代码。哪怕你一行不写,也得知道这堆东西是 Python 还是 JavaScript,出问题时才好对着问 AI、找对应的运行环境。

常见坑 新手以为得先精通某门语言才能开始。其实 vibecoder 阶段重点是看懂大概、会让 AI 改,而不是背语法。另一个坑是不同语言不能混着跑,Python 代码不能丢进网页直接用。

框架

库

代码编辑器 /

库 Library

就像做饭时直接买现成的火锅底料,不用自己从花椒辣椒一样样炒。别人把常用功能(比如算日期、发邮件、处理图片)提前写好打包,你拿来直接用,省得自己造。

价 值 几乎所有常见功能都有人写好了。你要的功能装个库一行调用就行,不用从零写几百行。这是现代编程能快的核心原因——站在别人肩膀上。

何时用到 让 AI 加个功能,比如'把日期显示成几天前',AI 通常会引入一个现成的库来做。你看到代码里 import 或 require 某个陌生名字,八成就是在用库。

常见坑 装太多库会让项目变臃肿、容易出冲突。还有库会过期或停止维护,带安全漏洞。别看到啥库都装,能用框架自带功能就别额外引。

框架

SDK

包/依赖

框架 Framework

库是你想用就用的调料,框架更像一套精装修的毛坯房:墙、水电、户型都给你定好了,你只往里填家具。它规定了整个项目怎么搭、文件放哪、流程怎么走。

价 值 从零搭一个网站要操心特别多杂活(怎么响应用户点击、怎么连数据库、怎么管页面跳转)。框架把这些套路都封装好,你按它的规矩填内容就能快速出成品,不用每次重造轮子。

何时用到 做网站常听到的 React、Vue、Next.js,做后端的 Django、Express,都是框架。你让 AI'用 React 做个落地页',它就是在框架的规矩里给你填代码。选错框架后面很难换。

常见坑 新手分不清框架和库:库是你叫它,框架是它叫你(它定流程,你填空)。另一个坑是框架学习曲线陡,且一旦选定很难中途换,所以技术选型时要慎重。

库

技术选型

编程语言

包/依赖 Package / Dependency

包就是别人打包好的一块功能(其实就是库的另一种叫法)。依赖是说'你的项目要正常跑,离不开这些包',就像做蛋糕依赖面粉和鸡蛋,缺一个就做不成。

价 值 你的项目几乎不可能全是自己写的,都站在一堆别人的包上面。把这些'依赖'清单记下来,换台电脑或上线时一键就能把同样的包全装回来,保证到哪都能跑。

何时用到 AI 生成的项目里通常有个叫 package.json(JS)或 requirements.txt(Python)的文件,列着所有依赖。报错'找不到某个模块',基本就是某个依赖没装上。

常见坑 最经典的坑:在自己电脑能跑,换台机器就崩,因为依赖没装全或版本对不上。别手动一个个装,照清单文件批量装。还有依赖套依赖,一个包能牵出几百个,体积爆炸很正常。

包管理器

库

SDK

包管理器 Package Manager

就像手机的应用商店:你说要装哪个 App,它帮你下载、装好、以后还能一键更新和卸载。包管理器就是代码世界的应用商店,帮你管所有要用的包。

价 值 手动下载、解压、放对位置、处理版本冲突会累死人。包管理器一条命令搞定全部:装包、升级、按清单一键还原整个项目的依赖。没它现代开发寸步难行。

何 时 用 到 JavaScript 用 npm(或 pnpm、yarn),Python 用 pip(或 uv)。AI 让你在终端敲 'npm install' 或 'pip install',就是在用包管理器装项目要的东西。这是几乎每个项目第一步。

常 见 坑 不同语言用不同的包管理器,别拿 npm 去装 Python 的包。还有网络问题导致装不上很常见(尤其国内),经常要配镜像源。命令敲错地方(不在项目文件夹里)也会失败。

包/依赖

终端 / 命令行

编程语言

技术栈 Tech Stack

就像开一家餐厅要凑齐一套设备:灶台、冰箱、收银机、外卖平台。做一个软件也要凑齐一套技术——用什么语言、什么框架、什么数据库、放在哪上线。这一整套搭配就叫技术栈。

价 值 一个完整产品从不靠单一技术,而是好几样配合。把这套搭配理清楚,你才知道项目由哪些部分组成、出问题该往哪查、招人或问 AI 时该怎么描述。

何 时 用 到 你跟人(或 AI)说'我这项目是 React + Node + Postgres',对方立刻就懂你的架子长什么样。看一个开源项目、评估一个外包方案,第一件事就是看它的技术栈。

常 见 坑 新手容易追最新最酷的技术堆栈,结果生态不成熟、出问题没人答。对 vibecoder,选主流、AI 熟悉、社区资料多的技术栈,远比选'先进'的重要。

技术选型

框架

数据库相关

技术选型 Tech Stack Selection

就像装修前定主材:地板选实木还是瓷砖、墙刷漆还是贴砖。一旦定下大动起来很费劲。技术选型就是在动手前决定用哪门语言、哪个框架、哪个数据库——后面整个项目都建在这之上。

价值 选对了,后面顺风顺水;选错了,做到一半发现这条路走不通,推倒重来代价极大。选型本质是权衡:好不好上手、社区大不大、贵不贵、AI 熟不熟、能不能撑住未来的量。

何时用到 开一个新项目的第一步。让 AI'帮我做个能收款的小程序',它会先建议一套技术方案——这就是在帮你做技术选型。你需要判断它的建议是否主流、好维护。

常见坑 vibecoder 最容易掉进'选最新最潮'和'选我听过名字的'两个坑。正解是问 AI'哪个方案资料最多、最不容易踩坑',选成熟主流的。冷门技术 AI 也答不好,等于自断外援。

技术栈

框架

开源

开源 Open Source

就像一道公开的菜谱:任何人都能免费看、照着做、还能改成自己的口味再分享出去。开源软件就是把代码完全公开,谁都能免费用、能看里面怎么写的、能拿来改。

价值 你做项目用到的绝大多数框架、库、工具几乎都是开源且免费的,省下天价授权费。代码公开还意味着全世界的人帮着挑 bug、补漏洞,通常比闭门造的更靠谱。

何时用到 你在 GitHub 上找到一个现成功能直接拿来用、AI 给你引入某个免费库、或想看某个工具到底怎么实现的——都在受益于开源。vibecoder 几乎全程站在开源之上。

常见坑 免费不等于随便用:每个开源项目有许可证(License),有的允许商用、有的要求你也得开源、有的禁止商用。拿别人代码做收费产品前,务必看清许可证,否则可能侵权。

GitHub / 代码仓库

库

框架

代码编辑器 / IDE Code Editor / IDE

写代码就像写文章,得有个趁手的写字软件。普通记事本能写但很难用;代码编辑器像专业版 Word,会自动给代码上色、提示拼写、还能直接运行——IDE 则是更全能的版本,连调试、装插件都包了。

价 值 好的编辑器让你写代码不痛苦:语法高亮看得清、自动补全少打字、报错当场标红。现在很多编辑器还内置 AI,你用大白话说需求它直接帮你改代码,这正是 vibecoder 的主战场。

何时用到 你打开项目文件夹改东西、跟 AI 一起写代码,都在编辑器里进行。常见的有 VS Code(最主流免费)、Cursor(主打 AI 编程)。你看到的代码界面基本就是它。

常见坑 新手纠结选哪个编辑器纠结半天。其实对 vibecoder 直接上 Cursor 或带 AI 的 VS Code 就行,先用起来。另一个坑是装一堆插件拖慢速度,够用就好。

终端 / 命令行

调试

编程语言

终端 / 命令行 Terminal / CLI

平时你用鼠标点图标办事,终端是反过来——用打字给电脑下命令。就像在微信里发指令而不是点按钮。一个黑框框,你敲一行字回车,电脑就执行,没有花哨界面。

价 值 很多开发操作没有按钮可点,只能在终端敲命令:装依赖、启动项目、上传代码到 GitHub 都靠它。看着原始,但比点来点去快,也是 AI 让你执行操作时最常用的地方。

何时用到 AI 让你'在终端运行 `npm install`'敲 `git push`',你就得打开那个黑框框照着粘贴执行。装环境、跑项目、部署上线几乎都绕不开它。

常见坑 新手怕黑框框,其实大多数时候就是复制 AI 给的命令粘进去回车。真正的坑是:命令要在对的文件夹里敲(不然找不到项目),以及复制命令前看一眼别盲粘危险操作(比如删除)。

包管理器

Git

代码编辑器 /

版本控制 **Version Control**

就像游戏的存档功能:每改一段就存个档,后悔了能随时读回任意一个存档点。版本控制帮你给代码不停存档,记录每一次改了什么,改崩了一键回到上一个好的版本。

价 值 没它,改崩了只能干瞪眼或手动撤销到吐血。有它,任何时候都能回退、能看清每次改动、能多人同时改同一个项目不打架。这是专业开发的命根子,vibecoder 也需要。

何 时 用 到 让 AI 一通改之后发现搞砸了,如果之前存了档(提交过),一条命令就能退回去。多人协作、想试个大胆改动又怕翻车时,版本控制就是你的后悔药。

常 见 坑 新手要么完全不用(改崩无法挽回),要么用了但从不'存档提交',等于白装。养成改完一段就提交一次的习惯,存档点越密,后悔成本越低。

Git

GitHub / 代码仓库

提交/推送

Git **Git**

Git 就是目前最流行的那台'存档机器'。版本控制是概念(给代码存档这件事),Git 是实现它的具体工具,就像'记账'是概念、'记账 App'是工具。几乎所有人都用 Git 来管代码版本。

价 值 Git 帮你把每次改动存成档、记录谁在何时改了什么、随时回退、还能把不同人的改动合并到一起。它是版本控制事实上的标准,学开发绕不过去。

何 时 用 到 你看到的 git commit、git push、git pull 这些命令都是 Git。把代码同步到 GitHub、和别人协作、给项目存档全靠它。vibecoder 至少要会提交和推送两个基本动作。

常 见 坑 Git 命令多且容易绕晕,新手常把自己搞到一团乱(尤其'合并冲突')。对 vibecoder,先掌握 add/commit/push 三件套就够,遇到复杂情况直接问 AI 怎么解,别硬扛。

版本控制

GitHub / 代码仓库

提交/推送

GitHub / 代码仓库 GitHub / Repository

如果 Git 是你电脑里的存档机, GitHub 就是放在云端的'网盘+协作平台', 把你的代码存档传上去。仓库(Repository)就是云端里专门放某个项目的一个文件夹, 装着这个项目的全部代码和历史。

价 值 代码传到 GitHub 上, 换电脑能拉下来、电脑坏了不丢、能邀请别人一起改、还能展示给全世界看。它也是全球最大的开源代码集散地, 你用的无数免费工具都托管在这。

何 时 用 到 想把项目备份上云、和人协作、或把代码交给部署平台自动上线, 第一步都是建个 GitHub 仓库。找现成开源项目、看 AI 工具的源码, 也都是去 GitHub。

常 见 坑 最危险的坑: 把密钥、密码不小心提交到公开仓库, 等于贴到全世界面前, 被人盗刷。一定要把含密钥的文件加进忽略清单(.gitignore)。另外公开和私有仓库要分清, 商业代码别设公开。

Git

提交/推送

开源

提交/推送 commit / push

提交(commit)是按下'存档'键, 把这次改动在本地存成一个档点, 还附上一句话说明改了啥。推送(push)是把本地这些档点'上传'到 GitHub 云端。先存档, 再上传, 两步。

价 值 提交让你的改动有了一个可回退的记录点, 推送让它同步到云端被备份和共享。这两是 Git 日常用得最多的动作, vibecoder 必会的最小操作集。

何 时 用 到 改完一段满意了就提交一次(存档), 想备份上云或让别人看到就推送。AI 帮你写完功能后, 常会提示你'提交并推送', 你照做代码就上 GitHub 了。

常 见 坑 新手常忘记提交, 辛苦的改变没存档, 一崩全没。还有人提交了忘推送, 以为上云了其实还在本地, 换电脑发现啥都没有。提交要写清说明, 别一律写'update', 以后自己都看不懂。

Git

GitHub / 代码仓库

分支

分支 Branch

就像写小说时复制一份草稿去试不同的结局,改坏了不影响正式稿,满意了再合并回去。分支就是从主线代码岔出一条独立线,在上面随便折腾,不碰正在用的稳定版本。

价值 想加新功能或做大胆改动又怕弄坏现有的东西?开个分支单独搞,试成了合并回主线,试砸了直接扔掉,主线毫发无伤。这让更多人能各干各的互不干扰,也让你敢于试验。

何时用到 你要给上线的网站加个新功能,但不想改坏正在跑的版本,就新建一个分支开发,验证没问题再合并。团队协作里每人一个分支是标配。

常见坑 vibecoder 一个人做小项目时,常常用不到分支,在主线上改也行,别被吓住。真用到时最大的坑是'合并冲突'——两条线改了同一行,合并时要手动取舍,遇到直接问 AI 怎么处理。

Git

提交/推送

CI/CD 持续集成与部署

CI/CD 持续集成与部署 CI/CD

就像一条自动化流水线:你把改好的代码往 GitHub 一推,流水线自动接手——检查代码有没有错、跑一遍测试、没问题就直接帮你发布上线。省得你每次手动重复一堆动作。

价值 手动上线要敲一长串命令、容易漏步骤、还可能把出错的代码发上去。CI/CD 把这些自动化:推一下代码,几分钟后新版本自己就上线了,又快又少出错。这是现代项目能频繁更新的关键。

何时用到 用 Vercel、Netlify 这类平台时,你把代码连上 GitHub,以后每次推送它就自动重新部署,网站自动更新到最新——这背后就是 CI/CD。vibecoder 用这些平台基本无感享受它。

常见坑 新手会被 CI/CD 这名字吓到以为很高深。其实用 Vercel 这类平台,你啥都不用配就自动有了。真要踩坑是:推了有 bug 的代码,流水线会忠实地把 bug 也发上线,所以推之前最好先验一下。

提交/推送

GitHub / 代码仓库

测试

Bug Bug

就是程序里的毛病、不按预期来的地方:点了按钮没反应、显示的数字算错了、页面突然白屏。就像菜里吃出根头发——本不该有却出现了。Bug 这词来历就是早年电脑里真卡进了一只虫子。

价 值 认识到 bug 是常态而非失败,心态就稳了。再厉害的程序员写的代码也有 bug,做软件本质就是不停发现和修 bug 的过程。能清楚描述一个 bug,是让 AI 帮你修好它的前提。

何 时 用 到 你做的功能没按预期工作时,就是遇到 bug 了。这时要做的是:看清错在哪、复现它(怎么操作能让它再出现一次)、然后把现象原原本本告诉 AI 让它帮你找原因。

常 见 坑 新手描述 bug 太含糊("它坏了"不行了"),AI 没法帮你。要说清:我做了什么、期望什么结果、实际出现了什么(最好带上报错原文)。报错信息别直接关掉,那是最值钱的线索。

调试

测试

代码编辑器 /

调试 Debug

就像家里灯不亮了,你一步步排查:是灯泡坏了?开关坏了?还是跳闸了?调试就是程序出毛病后,顺着线索一点点找出到底哪里错了的过程,而不是瞎改一通。

价 值 光知道有 bug 没用,得找到病根才能修。调试就是定位问题的手艺:在哪一步出的错、为什么出错。这是写代码最花时间的部分,也是最能体现功力的地方。

何 时 用 到 功能不对劲时,你打开报错信息、看看程序运行到哪卡住了、把出问题的代码和报错一起喂给 AI 让它分析——整个过程就是调试。vibecoder 调试的主力方式就是和 AI 一来一回排查。

常 见 坑 新手最大的坑是不看报错就让 AI'随便改改试试',改了一堆还是错。正解是先复制完整报错给 AI。另一个坑是一次改太多处,出错了都不知道是哪处导致的,要一次只改一个地方。

Bug

代码编辑器 /

测试

测试 Test

就像菜做好了先自己尝一口咸淡再端给客人。测试就是写一些'自动尝味道'的代码,提前检查你的功能对不对,而不是等用户用出问题才发现。

价值 人工每次都把所有功能点一遍很累且容易漏。自动化测试帮你在每次改动后一键检查'原来好的有没有被改坏了',让你敢放心地改代码、上线前心里有底。

何时用到 项目大了、改动频繁时,测试能帮你守住底线:改了 A 功能,跑一遍测试确认没把 B 功能带崩。配合 CI/CD,每次推代码自动跑测试,有问题当场拦下。

常见坑 vibecoder 早期小项目可以不写正式测试,手动点一遍核心流程也够,别为测试而测试。但'完全不验证就上线'是大坑——至少自己把主要功能走一遍。让 AI 帮写测试时也要核对它测得对不对。

Bug

调试

CI/CD 持续集成与部署

SDK SDK (Software Development Kit)

就像买宜家家具送的一整套工具包:螺丝、扳手、说明书全配齐,让你能顺利把家具装起来。SDK 是某个服务方给你的一整套现成工具+说明,让你能轻松把他们的功能接进你的项目。

价值 想接入微信支付、调用某个 AI 模型、用某个云服务?对方通常提供 SDK,把复杂的对接细节都打包好,你照着几行代码就能用上他们的能力,不用自己啃一堆底层文档。

何时用到 接支付('装微信支付 SDK')、接 AI('用 OpenAI 的 SDK 调模型')、接地图定位等等,第一步往往是装对方的 SDK。AI 帮你集成第三方服务时,基本都在用人家的 SDK。

常见坑 新手常分不清 SDK、库、API:SDK 是一整套工具包(里面可能就含库),API 是对方开放的接口(SDK 是帮你方便调用 API 的工具)。另一个坑是 SDK 版本更新后用法会变,照着过时教程会对不上。

库

REST API

包/依赖

JSON 数据格式 JSON

就像填一张标准表格:姓名填什么、年龄填什么,一栏一栏清清楚楚。JSON 是一种程序之间传数据的通用格式,用'名称:值'成对地写,人能看懂、机器也好处理,是数据界的普通话。

价 值 两个程序要交换数据,得用双方都认的格式,JSON 就是最通用的那个。你调任何 API、看配置文件、存数据,几乎都会碰到 JSON。看懂它,你就能读懂程序间传的到底是什么。

何 时 用 到 调用任何接口拿数据回来,返回的基本都是 JSON。项目里的 package.json 配置文件也是 JSON。AI 给你看接口返回结果时,那一坨带大括号、引号的东西就是 JSON。

常 见 坑 JSON 格式很较真:少个逗号、多个逗号、引号用错、括号没配对,整个就报错。新手手写 JSON 最常死在标点上。复制粘贴时也容易带进多余字符。让 AI 检查格式是最省事的办法。

REST API

Webhook 回调

包/依赖

REST API REST API

就像餐厅点菜:你(你的程序)按菜单给服务员(API)下单'我要一份天气数据',后厨(对方服务器)做好用标准方式端回给你。REST 就是这套点菜规矩的一种通用约定,大家都按它来,谁都能对接谁。

价 值 想用别人的能力(查天气、查物流、调 AI)却又不能进人家系统,REST API 就是对方开的一个'点菜窗口',你按规矩发请求,它返回数据。这是程序之间互相调用功能的最主流方式。

何 时 用 到 接入任何第三方数据或服务,十有八九是调它的 REST API。你让 AI'帮我接个查快递的功能',它就是在帮你按 REST 规矩去请求快递公司的 API。

常 见 坑 调 API 通常要密钥(身份凭证)且常常按调用次数收费,新手容易忘看收费规则,被刷爆账单。密钥也绝不能写死在前端或传到公开仓库,会被人盗用。还要做好对方返回错误时的处理。

JSON 数据格式

SDK

WebSocket

Webhook 回调 **Webhook**

普通 API 是你主动打电话去问'到货了吗';Webhook 反过来,是货到了对方主动打电话通知你。你提前给对方留个电话号(一个网址),事情一发生,对方就主动推消息过来,你不用一直追问。

价 值 有些事你不知道啥时候发生(用户付款成功了没、订单状态变了没),一直去问既费劲又慢。Webhook 让对方在事情发生的那一刻主动通知你,实时又省力。支付、消息系统全靠它。

何 时 用 到 接支付时,用户付完款,支付平台通过 Webhook 通知你'这笔到账了',你才好发货/开通服务。接收消息、自动化流程(如某事件触发后干啥)也大量用 Webhook。

常 见 坑 Webhook 是对方往你这发请求,所以你得有个能公开访问的网址接收(本地电脑收不到,要部署上线或用内网穿透工具)。还有安全坑:得验证消息真是对方发的,否则有人伪造'付款成功'骗你发货。

REST API

JSON 数据格式

CI/CD 持续集成与部署

WebSocket **WebSocket**

普通 API 像发短信:问一句答一句,每次都要重新拨号。WebSocket 像打电话一直不挂:连接一旦建立就保持着,双方随时能说话。所以聊天、实时弹幕、在线协作才能即时收到对方的消息。

价 值 需要实时双向交流的场景(聊天室、直播弹幕、多人协作、AI 一个字一个字往外蹦答案),用普通 API 反复请求又慢又费。WebSocket 保持长连接,消息一有就立刻送达,实时性拉满。

何 时 用 到 做聊天功能、实时通知、协作白板,或想让 AI 回答像打字机一样逐字显示(流式输出),底层常用 WebSocket。vibecoder 用现成的实时服务时多半也是它在背后撑着。

常 见 坑 WebSocket 比普通 API 复杂:连接会断要会自动重连、用户多了服务器扛连接压力大、调试也更难。新手别一上来就自己【原文缺字】framework,能用现成的实时服务(如某些云服务)就别自己造。

REST API

Webhook 回调

JSON 数据格式

ORM 对象关系映射 ORM

数据库存数据用的是它自己的一套'方言'(SQL 查询语句)。ORM 像个翻译官:你用熟悉的编程语言说'给我这个用户的订单',它自动翻译成数据库的方言去取,再把结果翻译回来,你不用学那套方言。

价 值 直接写数据库查询语句既要学新语法、又容易写出有安全漏洞的代码。ORM 让你用平时写代码的方式操作数据,省心又相对安全,是后端开发常用的省力工具。

何 时 用 到 项目要存用户、订单、文章这类数据到数据库时,AI 通常会用一个 ORM(如 Prisma、SQLAlchemy)来读写,而不是手写一堆原始 SQL。你看到代码里用对象方法增删改查数据,多半是 ORM。

常 见 坑 ORM 方便但会'藏'掉底层效率问题:一个看似简单的操作可能偷偷查了数据库几百次,数据量大了就卡。还有复杂查询 ORM 写起来反而别扭。新手知道有这回事即可,卡顿时记得往这方面查。

JSON 数据格式

技术栈

REST API

AI 与大模型

04

vibecoder 的核心生产力,和 AI 打交道的语言

20 词

人工智能 Artificial Intelligence (AI)

就像一个学了海量知识的超级实习生:你给它一个任务,它能自己琢磨出该怎么做,而不是只会按你写死的步骤一步步执行。区别在于它能"举一反三",遇到没见过的情況也能给个像样的答案。

价 值 对 vibecoder 来说,AI 是把"想法"变成"能跑的网站/小程序"的发动机。你不用懂代码语法,用大白话描述需求,AI 帮你写、帮你改、帮你查错,把原本需要专业程序员几周的活压缩到几小时。

何 时 用 到 你做的几乎每件事背后都是 AI:让 Cursor/Claude 帮你写一个登录页、生成一段支付接入代码、解释一个报错、把网站文案润色一遍。它是你这一整层所有工具的总称。

常 见 坑 以为 AI 什么都对、什么都会。它本质是"很会猜下一句"的程序,自信地说错话是常态(见幻觉)。重要的事(支付金额、用户隐私、删数据)必须自己核一遍,不能闭眼信。

机器学习

大语言模型

Agent 智能体

大语言模型 Large Language Model (LLM)

想象一个读完了大半个互联网的人,你说上半句他能接下半句。它的核心本事就是"根据前面的文字,猜出最合适的下一个字",一个字一个字地往外蹦,蹦出来的就是一整段通顺的回答。

价 值 它是你天天对话的那个"AI"的真身。ChatGPT、Claude、豆包、DeepSeek 背后都是 LLM。你写代码、问问题、让它改文案,实际都是在跟一个 LLM 聊天,它是 vibecoder 最核心的生产力工具。

何 时 用 到 打开 Cursor 让它写代码、问 ChatGPT 一个报错怎么解、让 Claude 把需求拆成步骤——只要你在跟 AI 用自然语言对话,你就是在用 LLM。选哪个模型(GPT、Claude、Gemini)直接决定写代码的质量。

常 见 坑 别把它当数据库或计算器。它是"猜字高手"不是"查证专家",算数、记具体数字、引用真实链接都可能出错。需要准确事实时,要么让它联网查,要么自己验。

人工智能

Token 词元

上下文 / 上下文窗口

幻觉

提示词 Prompt

就是你对 AI 说的那句话/那段话,相当于给实习生派活时的"任务说明"。你说得越清楚(要做什么、给谁看、什么风格、有什么限制),它干得越对;你说得含糊,它就只能瞎猜。

价 值 提示词是 vibecoder 唯一的"操作杆"。你不写代码,你写的是提示词——一句话能让 AI 生成整个页面,也能让它把代码改崩。会写提示词,基本等于会指挥 AI 干活。

何 时 用 到 无时无刻:让 AI 写一个"带手机号登录的注册页"、让它"把这个按钮改成蓝色圆角"、让它"找出这段代码为什么报 404"。你打字框里敲的每一句都是提示词。

常 见 坑 最常见的坑是太笼统,比如只说"做个网站"。AI 会自己脑补一堆你没要的东西。要给背景、给例子、给约束("只改这个文件,别动别的"),否则返工无穷。

提示工程

大语言模型

上下文 / 上下文窗口

Token 词元 Token

AI 不是按"字"或"词"来读文字的,而是把文字切成一小块一小块的"碎片",每块叫一个 token。英文里大概 4 个字母算一个 token,中文里通常一个汉字就是一两个 token。AI 读和写,都是一块一块来的。

价 值 Token 是 AI 的"计价单位"和"容量单位"。你调用 AI 接口要按 token 数付费(发进去的 + 吐出来的都算钱),AI 一次能处理多少内容也是按 token 数封顶。理解它才能算清成本、避免超限。

何 时 用 到 当你发现 AI 突然"忘了"前面说的话(token 超了上下文上限),或者月底收到一笔比预想高的 API 账单(token 用太多)时,就会真切体会到 token 的存在。

常 见 坑 以为"几句话"花不了多少。其实把一整个代码文件、一篇长文档塞给 AI,token 蹭蹭涨,费用和速度都受影响。中文比英文更费 token,别忽略。

上下文 / 上下文窗口

大语言模型

API调用 vs 本地部署

上下文 / 上下文窗口 **Context / Context Window**

上下文就是 AI 当前对话能"记住"的全部内容,像一张有固定大小的桌子。你和它聊的每句话、贴进去的每段代码都摆在这张桌子上;桌子满了,最早摆上去的就被挤下去——它就"忘"了开头说过什么。

价 值 上下文窗口的大小,决定了 AI 一次能"看懂"多大的项目。窗口大,你能把整个代码库、整份需求文档一起喂给它,它改起来才不会顾此失彼;窗口小,它就只能瞎子摸象。

何 时 用 到 做一个稍大的项目让 AI 帮忙改代码时,如果它改着改着开始"失忆"、重复犯刚纠正过的错、忘了你定的规则,基本就是上下文窗口被塞满了。

常 见 坑 误以为它能永远记住整段对话。一旦超出窗口,早期内容静默丢失,它不会提醒你。长对话/大项目里要主动把关键约束重新贴一遍,或开新对话重新交代。

Token 词元

提示词

大语言模型

提示工程 **Prompt Engineering**

如果说提示词是"对 AI 说的一句话",提示工程就是"把这句话说到位的手艺"——好比同样请人帮忙,会说话的人三言两语就把事办成,不会说的人来回扯皮还办砸。它是一套让 AI 听话、少出错的说话技巧。

价 值 对不写代码的 vibecoder,这是性价比最高的技能。同一个 AI,提示工程好的人能让它一次写对、少返工;不会的人反复改还得不到想要的结果。它直接决定你用 AI 的产出质量和速度。

何 时 用 到 当你发现"为什么别人用同一个 AI 做得又快又好,我却老出问题"时,差距通常就在提示工程:给角色设定、给示例、分步骤、明确约束、让它先想再做。

常 见 坑 以为有什么"万能咒语"复制粘贴就行。真正有用的是把任务讲清楚 + 给例子 + 限定范围。盲目堆砌"你是世界顶级专家"这类空话,作用很小。

提示词

大语言模型

幻觉

幻觉 Hallucination

就像一个不肯说"我不知道"的人:你问他一个他不确定的事,他不会沉默,而是一本正经地编一个听起来很合理的答案。AI 也一样,它的本能是"把话接下去",哪怕是瞎编,也说得理直气壮。

价 值 认清幻觉,是 vibecoder 不翻车的底线。知道 AI 会自信地胡说,你才会在关键处(支付逻辑、数据库操作、引用的链接和数字)留个心眼去核实,而不是闭眼上线。

何 时 用 到 AI 给你一个根本不存在的函数、引用一个打不开的网址、信誓旦旦报一个错误的价格或数字、编造一个其实没有的软件功能——这些都是幻觉,在写代码时极常见。

常 见 坑 最危险的是它编得太像真的。代码能跑不代表逻辑对,链接像模像样不代表能打开。涉及钱、数据、安全的部分,务必自己跑一遍、点一遍、查一遍,别信 AI 的口头保证。

大语言模型

RAG 检索增强生成

推理

API调用 vs 本地部署 API Call vs Local Deployment

用 AI 有两种方式。API 调用像"叫外卖":你不用自己有厨房,网上下单(发请求),对方做好(AI 算好)送回来,按份付钱。本地部署像"自己家装了厨房":把 AI 模型下载到自己电脑/服务器跑,不用联网、不按次付费,但要自己买设备、自己维护。

价 值 这是 vibecoder 接入 AI 的第一个岔路口。99% 的情况下你用 API 调用——简单、免运维、随时用最强模型。本地部署只在你特别在意数据隐私、想省长期费用、或要离线用时才考虑。

何 时 用 到 给你的网站/小程序接入 AI 功能(比如智能客服、自动写文案)时,你要在代码里填一个 API key 去调用 OpenAI/Claude/国产模型——这就是 API 调用。除非有特殊需求,新手别碰本地部署。

常 见 坑 新手常误以为本地部署"免费又安全"就一头扎进去。实际它要不便宜的显卡、要会配环境、效果还往往不如直接调最新闭源模型。先用 API 把产品跑通,有真实需求再说。

开源模型 vs 闭源模型

推理

Token 词元

Agent 智能体 AI Agent

普通 AI 像个只会聊天的顾问:你问一句它答一句。Agent 像个能跑腿的助理:你给它一个目标("帮我把这个 bug 修了"),它会自己拆解步骤、动手查文件、改代码、跑一遍看对不对、不对再改,一直忙到把活干完才回来交差。

价 值 Agent 是 vibecoder 的"自动化升级版"。它把"你说一步它做一步"变成"你说目标它全包",能连续操作十几步而不用你盯着。Cursor、Claude Code 这类工具的核心能力就是 Agent。

何 时 用 到 让 Cursor"自动把整个项目跑起来并修复所有报错"、让 Claude Code"按这个需求新建几个页面并接好接口"——它会自己读文件、写代码、执行命令、循环验证,这就是 Agent 在干活。

常 见 坑 它会自主操作,所以也会自主闯祸:可能改了你没让它改的文件、删了不该删的东西。让它放手干之前,代码最好先用 git 存档,关键操作让它先汇报再执行,别全程放养。

MCP 模型上下文协议

大语言模型

提示词

MCP 模型上下文协议 Model Context Protocol (MCP)

MCP 像是给 AI 配的一个“万能插座标准”。以前每接一个新工具(数据库、网盘、企业系统)都要单独焊根电线,又乱又费劲。有了 MCP 这个统一插座,任何工具做成 MCP 插头,AI 一插就能用,不用每个都重新对接。

价值 它让你的 AI 助手从“只会聊天”变成“能连真实世界干活”。装个 MCP 服务,Claude/Cursor 就能直接读你的数据库、查企业工商信息、操作你的文件——不用自己写一堆胶水代码去打通。

何时用到 想让 AI 编程助手直接读你的项目数据库、调某个第三方服务、或接入一个现成能力(如网页抓取、企业查询)时,装一个对应的 MCP 服务比自己写对接代码省事得多。

常见坑 由 Anthropic 在 2024 年 11 月推出、2025 年底捐给 Linux 基金会旗下基金会托管,生态新、服务质量参差。装第三方 MCP 等于给 AI 开了访问权限,来路不明的别乱装,小心数据泄露。

Agent 智能体

API调用 vs 本地部署

大语言模型

RAG 检索增强生成 Retrieval-Augmented Generation (RAG)

普通 AI 答题靠“脑子里记的”,可能记错或没学过。RAG 像“开卷考试”:回答前先去你指定的资料库里翻一翻(检索),把找到的相关内容垫在手边,再照着这些真实资料组织答案。这样它就不容易瞎编了。

价值 RAG 是让 AI“懂你自己的资料”的标准做法。想做一个能回答“我们公司产品手册”“我这堆笔记”的智能问答,而这些内容 AI 原本不知道,RAG 让它现查现答,既准确又能附出处。

何时用到 给网站做一个“基于我们帮助文档的智能客服”、做一个“能问我个人知识库的助手”时,用的就是 RAG:把你的文档切块存起来,用户提问时先检索相关段落再让 AI 作答。

常见坑 以为 RAG 就万无一失。它的答案质量取决于“检索得准不准”:资料没整理好、切块切得乱,AI 翻到的就是无关内容,照样答错。垃圾进、垃圾出,资料质量是关键。

Embedding 嵌入向量

向量数据库

幻觉

Embedding 嵌入向量 Embedding

把一句话/一段文字翻译成一长串数字坐标,意思相近的内容,坐标也挨得近。好比给城市标经纬度——"北京"和"天津"坐标接近,和"巴黎"就很远。这样电脑就能用"算距离"的方式,判断两段文字意思像不像。

价 值 它是"按意思搜索"的底层魔法。传统搜索只能匹配一模一样的关键词,Embedding 让 AI 能找到"意思相近但用词不同"的内容,是 RAG、智能问答、相似推荐这些功能能成立的基础。

何 时 用 到 做 RAG 智能问答时,系统在背后把你的文档和用户问题都转成 Embedding,再比对谁离得近来找答案。你一般不直接操作它,但它是"AI 能读懂你资料"的隐形地基。

常 见 坑 新手容易和"AI 生成回答"搞混。Embedding 不负责答题,它只负责"把文字变成可比较的坐标"。另外不同模型生成的 Embedding 不通用,中途换模型得把资料全部重新转一遍。

向量数据库

RAG 检索增强生成

大语言模型

向量数据库 Vector Database

普通数据库像"按身份证号精确查人"的档案柜,要求查得分毫不差。向量数据库像"按长相找相似的人"的相册:你给它一张脸,它能从一百万张里飞快挑出最像的几张。它专门存 Embedding 那串坐标,擅长"找最接近的"。

价 值 它是 RAG 应用的"资料仓库"。当你把成千上万段文档转成 Embedding 后,得有个地方存起来并能秒速找出"和用户问题最相关的几段",向量数据库就干这个,是智能问答能快速响应的关键。

何 时 用 到 做一个基于自有文档的 AI 问答、AI 客服、相似内容推荐时,你会用到向量数据库(如 Pinecone、Milvus、或带向量功能的 PostgreSQL)来存检索资料。规模小时甚至一个文件就够。

常 见 坑 小项目盲目上专业向量数据库,徒增复杂度和费用。几百几千条数据,用现成的轻量方案或数据库插件就够。先别为还没到的规模过度设计。

Embedding 嵌入向量

RAG 检索增强生成

API调用 vs 本地部署

多模态 Multimodal

早期 AI 像只认字的人,只能读文字、回文字。多模态 AI 像五官齐全的人:能看图、能听声、能看视频,还能边看边说。你拍张照片问它"这是什么菜怎么做",它真能看懂图再回答。

价 值 它把 AI 能干的活从"文字"扩展到"看和画"。vibecoder 可以直接截一张设计图丢给 AI 让它生成对应代码、拍张报错截图让它帮你诊断、或让它生成产品配图——不用全靠打字描述。

何 时 用 到 把一张网页设计稿截图发给 Cursor 让它"照着这个做出来"、把报错界面截图发给 AI 问"这是什么问题"、让 AI 给你的小程序生成图标和插画时,用的都是多模态能力。

常 见 坑 以为它"看图"和人一样精确。它能看个大概,但图里的小字、精细数据、复杂图表常会看漏或看错。让它读图里关键数字时,务必自己再核对一遍。

大语言模型

幻觉

人工智能

机器学习 Machine Learning

传统编程是人把规则一条条写死("如果邮件含'中奖'就标垃圾")。机器学习反过来:你给电脑看一万封"已标好是不是垃圾"的邮件,让它自己总结出规律。它不是被教规则,是从例子里自己学规则。

价 值 它是"AI 为什么会变聪明"的根本原理。理解这点,你就明白 AI 的能力全靠"喂的数据"——数据决定它擅长什么、会在哪犯错,而不是某个程序员把答案写死在里面。

何 时 用 到 作为 vibecoder 你基本不用自己做机器学习(那是算法工程师的活),但它是你理解整个 AI 生态的背景知识:为什么模型有"训练"和"使用"两个阶段、为什么微调能定制 AI。

常 见 坑 别和"大语言模型""AI"混为一谈、以为是新东西。机器学习是个用了几十年的大类技术,大语言模型只是它近年最火的一个分支。它也不是"会自己越用越聪明",得专门重新训练。

人工智能

大语言模型

微调

微调 Fine-tuning

通用大模型像个名校毕业的通才,什么都懂一点。微调像"岗前专项培训":再拿你这一行的专属资料集中训它一阵,让它说话风格、专业术语更贴合你的业务,从通才变成你的"专属老员工"。

价 值 当你需要 AI 稳定地用某种固定风格、格式或专业口径输出,光靠写提示词总不够稳时,微调能把这种偏好"焊"进模型里,让它每次都按你要的来,而不用每次都长篇大论地交代。

何时用到 实话说,vibecoder 早期几乎用不到。绝大多数定制需求,用好提示词 + RAG 就够了。只有当你有大量专属数据、且对输出风格/格式有严格统一要求时,才值得考虑微调。

常见坑 新手最大的坑是"一上来就想微调"。它要准备高质量数据、花钱训练、还可能把模型原有能力训坏。先穷尽提示词和 RAG,这俩解决不了再说微调,顺序别搞反。

机器学习

提示工程

RAG 检索增强生成

推理 Inference

模型有两段人生:"训练"是上学苦读、积累本事的阶段(很贵、很慢);"推理"是毕业上岗、实际干活回答你问题的阶段。你每次和 AI 对话、让它写代码,就是在让它"推理"——它在动用学到的本事现场作答。

价 值 理解推理,你才搞得清 AI 的钱花在哪、为什么有时候快有时候慢。你用 AI 的每一次请求都是一次推理,API 按推理消耗的 token 收费;模型越大、回答越长,推理越慢越贵。

何时用到 "推理"这个词你会在两处遇到:一是算成本(API 按推理用量收费);二是选模型时看到"推理模型"(指会先在心里想一长串再回答、更擅长复杂逻辑的那类,比普通模型慢但更准)。

常见坑 容易把"推理(模型干活)"和"训练(模型学习)"搞混。你作为用户接触的全是推理,从不碰训练。另外"推理模型"虽强但慢且贵,简单任务用它是杀鸡用牛刀,浪费钱和时间。

大语言模型

API调用 vs 本地部署

Token 词元

参数量 Parameters (7B / 70B)

参数就像 AI 大脑里的"神经连接",数量越多,脑容量越大、越聪明。7B 是 70 亿个,70B 是 700 亿个(B = Billion 十亿)。好比 7B 是个机灵的高中生,70B 是个博学的研究生——块头越大,通常越懂行,但也越能吃(占内存、跑得慢)。

价 值 参数量是你判断模型"大小档次"的快速标尺,尤其在考虑本地部署时直接决定"我的电脑跑不跑得动"。大模型更聪明但要好显卡、跑得慢;小模型笨一点但轻便、能塞进普通设备。

何时用到 主要在两个场景看它:一是想本地部署时,得按自己显卡内存挑跑得动的尺寸(7B 普通显卡能跑,70B 要专业设备);二是对比开源模型时,参数量是常见的规格标签。

常见坑 别迷信"参数越大越好"。新的小模型经常打败老的大模型,数据和训练方法比单纯堆参数更重要。而且大模型对硬件要求陡增,本地跑大模型,新手十有八九卡到崩溃。

开源模型 vs 闭源模型

API调用 vs 本地部署

推理

温度 Temperature

温度是 AI 回答的"放飞程度"旋钮。调低(接近 0),它每次都给最稳妥、最常规的答案,像照本宣科的老实人;调高,它更敢天马行空、用词更花,但也更容易跑偏、瞎编,像喝了点酒的段子手。

价 值 它让你能控制 AI 是"求稳"还是"求创意"。要它老老实实写代码、提取数据、按格式输出,就调低;要它帮你头脑风暴文案、起名字、想点子,就调高。一个旋钮切换两种风格。

何时用到 自己写代码调 API 时,你能在参数里设温度。让 AI 干"必须准"的活(改代码、整理数据)用低温;干"要发散"的活(想标语、起产品名、写营销文案)用高温。日常聊天界面通常已调好默认值。

常见坑 误以为"温度越高 AI 越聪明"——恰恰相反,高温会增加幻觉和出错。需要准确、可复现结果时,温度一定要调低。另外它只影响"措辞的随机性",不会让模型真的更博学。

大语言模型

幻觉

推理

开源模型 vs 闭源模型 Open-source vs Closed-source Models

闭源模型像"只能进店消费的米其林餐厅"(GPT、Claude、Gemini):菜做得顶级,但你只能点菜、按次付钱,进不了后厨、改不了配方。开源模型像"把菜谱免费公开的料理"(Llama、DeepSeek、Qwen):你能拿回家自己做、自己改,但得有厨房和手艺。

价值 这是你选"用哪个 AI"的核心分水岭。闭源通常最聪明、开箱即用,适合追求效果、省心;开源能免费下载自己跑、数据不出门、能深度定制,适合在意隐私、成本或要私有化的场景。

何时用到 给产品选 AI 大脑时拍板用哪类:大多数 vibecoder 直接调闭源模型的 API 最省心、效果最好;只有当你特别在意数据隐私、想长期省钱、或要私有部署时,才考虑下载开源模型自己跑。

常见坑 两个误区:一是以为"开源=免费",自己跑要算上设备和运维,小规模下往往比调闭源 API 还贵;二是以为开源一定差——到 2026 年顶尖开源模型(如 DeepSeek、Qwen)在很多任务上已逼近闭源,只在最难的推理和编程上还差一档。

API调用 vs 本地部署

参数量

大语言模型

安全与鉴权

05

不懂这层最容易出事:密钥泄露、被刷爆、数据被偷

15 词

API Key 密钥 API Key

就像进小区的门禁卡。你要用别人家的服务(比如让 AI 帮你画图、发短信),对方给你一张专属卡,刷卡才放行,还能记账算钱。卡丢了,捡到的人就能冒充你刷。

价 值 让你能调用别人的付费服务(OpenAI、地图、短信),对方靠它认出是你、给你权限、按你的用量收费。没有它,大部分第三方服务根本不让你用。

何 时 用 到 接入任何外部服务时第一步就是去对方网站申请一个 Key,粘到你代码里。用 AI 编程、接支付、调天气接口、发邮件,几乎都绕不开这一步。

常 见 坑 最大灾难是把 Key 写进代码后传到 GitHub 公开仓库,几分钟就被爬虫扫到盗刷,有人一夜被刷上万元。必须放进 .env 文件并加入 .gitignore,绝不进代码库。

.env 文件 / 密钥泄露

认证

速率限制

认证 Authentication Authentication

就是机场过安检查身份证,核对你到底是不是你本人。系统问的是同一个问题:你是谁?用账号密码、验证码或指纹来证明。

价 值 网站靠它确认来访者的真实身份,挡住冒充别人的人。没有认证,任何人输个名字就能进你账号,聊天记录、订单、钱包全裸奔。

何 时 用 到 只要你的网站或小程序有登录功能,就需要认证。让用户注册登录、记住谁是谁、显示对应的个人数据时,背后跑的就是认证。

常 见 坑 新手最容易把密码用明文直接存数据库,一旦库被偷所有人密码全泄露。密码必须先哈希再存。另外别自己手搓认证逻辑,优先用成熟服务,漏洞太多。

授权

Token / JWT 令牌

哈希 Hash(密码存储)

OAuth 第三方登录

授权 Authorization Authorization

认证是查身份证证明你是谁,授权是看你的门票决定你能进哪个厅。同样进了电影院,普通票只能进 3 号厅,VIP 票才能进包厢——你是谁已确认,接下来管你能干什么。

价 值 决定每个已登录用户能看什么、改什么。靠它区分普通用户和管理员,防止 A 用户偷看或篡改 B 用户的数据,是数据安全的核心闸门。

何 时 用 到 做后台管理、会员等级、付费内容时必用。比如免费用户不能用高级功能、普通员工看不到财务数据、用户只能改自己的资料,这些限制都是授权在管。

常 见 坑 最常见漏洞:接口只检查了你登录没,没检查这条数据是不是你的。结果把网址里的 id=123 改成 124 就能看到别人的订单。每次操作都要校验归属权。

认证

最小权限原则

Token / JWT 令牌

Token / JWT 令牌 Token / JWT

像游乐园入口换的手环。进门时验过身份(认证),给你戴个手环,之后玩每个项目只看手环不再查身份证。JWT 是一种手环,上面还印着你的会员等级和有效期,工作人员扫一眼就知道你能玩啥。

价 值 让你登录一次后,后续每次操作都带着令牌就行,不用反复输密码。服务器也不必每次查数据库,效率高,是现代登录态的标准做法。

何 时 用 到 用户登录成功后,服务器发一个 token 给浏览器存着,之后每次请求都自动带上。做前后端分离的网站、App、小程序登录时,几乎都在用这套。

常 见 坑 JWT 里的内容是编码不是加密,谁拿到都能解开看,别往里塞密码等敏感信息。还要设过期时间,token 被偷在过期前都能冒充你,所以别存在不安全的地方。

认证

授权

加密

OAuth 第三方登录 OAuth

就是网站上那个微信登录、用 Google 登录的按钮。好比去酒店不用押身份证原件,前台帮你刷一下就确认了身份,你的密码始终只交给微信,新网站永远看不到。

价 值 用户不用为每个网站记一套密码,点一下就用已有的微信/Google 账号登入,注册转化率高。你也不用自己存密码,把最危险的环节外包给大厂。

何 时 用 到 想让用户快速注册、降低门槛时接入。做面向 C 端的网站小程序时,微信登录几乎是标配;面向海外或开发者的产品常用 Google、GitHub 登录。

常 见 坑 OAuth 流程环节多、配置回调地址容易出错,新手自己实现易留漏洞。建议用 Supabase、Clerk、Auth0 这类现成服务,它们把第三方登录都封装好了,几行代码搞定。

认证

Token / JWT 令牌

API Key 密钥

哈希 Hash(密码存储) Hash

像把鸡蛋打成蛋花——能从蛋变蛋花,没法从蛋花还原成蛋。密码哈希后存的是那碗蛋花,你下次登录把输入的密码也打成蛋花,两碗对得上就放行,但谁偷走数据库也还原不出你原始密码。

价 值 让你存密码却不真的存密码,数据库即使被偷,小偷也拿不到明文密码。这是保护用户密码的底线,也是法律和合规的硬要求。

何 时 用 到 任何有注册登录、要存密码的功能都必须做。用户注册时把密码哈希后入库,登录时再哈希对比,全程数据库里没有一个明文密码。

常 见 坑 千万别用 MD5、SHA1 这些老算法,早被破解。2026 年首选 Argon2id(OWASP 推荐),bcrypt(成本因子设 12 以上)也仍安全。更省心的做法是用现成认证服务,它们默认就帮你哈希好了。

认证

加密

.env 文件 / 密钥泄露

加密 Encryption

像给信件上锁的保险箱。哈希是单向打碎不可还原,加密是双向的:有钥匙的人能锁上也能解开。你用对方的锁锁住内容,只有持钥匙的收件人才打得开,中途被截走也是一堆乱码。

价 值 保护数据在传输和存储中不被偷看。聊天内容、银行卡号、隐私信息加密后,即使被黑客截获或硬盘被偷,看到的也是乱码,拿不到真实内容。

何 时 用 到 网址前面的 https 就是传输加密,现在是标配。另外存身份证号、手机号等敏感字段进数据库前,也常先加密。接支付、做隐私功能时尤其要注意。

常 见 坑 新手常混淆加密和哈希:密码该用哈希(不可还原),不该用加密。另外加密的钥匙本身要保管好,钥匙泄露等于没加密;别把钥匙和密文存在一起。

哈希 Hash(密码存储)

Token / JWT 令牌

.env 文件 / 密钥泄露

.env 文件 / 密钥泄露 .env File / Secret Leak

就是你家所有钥匙(API Key、数据库密码)放在一起的钥匙盒,叫 .env。代码可以拿去给别人看,但这个钥匙盒绝不能跟着代码一起送出去——一旦上传到公开网络,等于把家门钥匙挂在大街上。

价 值 把密钥从代码里抽出来集中存到一个不会被上传的文件,代码可以公开分享,密钥留在本地。这是防止盗刷、数据被偷的第一道也是最关键的防线。

何 时 用 到 几乎每个项目都该有。凡是有 API Key、数据库密码、各种 secret,都写进 .env,并在 .gitignore 里把它排除。让 AI 帮你建项目时,主动要求它这么配置。

常 见 坑 头号事故就是密钥泄露:忘了加 .gitignore、把 .env 截图发群里、或粘贴到聊天工具,都可能被扫描盗用。发现泄露要立刻去对方平台作废旧 Key 重新生成,光删文件没用。

API Key 密钥

加密

最小权限原则

速率限制 Rate Limiting Rate Limiting

像游乐园热门项目限流:每人每小时最多玩 3 次。系统给每个用户或每个 IP 设个次数上限,超了就先等等,防止有人插队把资源占满。

价 值 挡住恶意刷量和暴力破解。没有它,坏人可以一秒发几千次请求猜你的密码、刷爆你的短信接口、或把你调用的付费 AI 接口刷出天价账单。

何 时 用 到 登录、发验证码、发短信、调用花钱的 API 这些环节都该加。做注册登录、接短信验证码、对外开放接口时,不限速就是在给攻击者开后门。

常 见 坑 新手常常完全不设限,直到收到天价账单或被刷爆短信费才后悔。尤其是发短信、调大模型这类一次就花钱的接口,必须从第一天就限速。

API Key 密钥

认证

防火墙

SQL注入 SQL Injection SQL Injection

数据库听指令查数据,你的输入框本该只填名字。但坏人在框里填一句暗藏的指令,系统没分清哪是数据哪是命令,就照着执行了——好比你让客服查订单,他却把你顺口说的'顺便把所有人资料发我'也照办了。

价 值 理解它才能防住:这是最经典、危害最大的攻击之一,一旦中招,整个数据库的用户资料、密码可能被一锅端走甚至删库。防住它是数据安全的基本功。

何 时 用 到 只要你的网站有输入框、搜索、登录,且后台要查数据库,就有这个风险。做任何需要用户输入再去查数据库的功能时,都要注意。

常 见 坑 根源是把用户输入直接拼进数据库查询语句。正确做法是用参数化查询(让数据库把输入只当数据,不当命令)。用主流框架并按规范写,基本能自动防住,别自己拼字符串。

XSS 跨站脚本

认证

防火墙

XSS 跨站脚本 XSS (Cross-Site Scripting)

网页本该只显示用户发的文字。但坏人在评论里塞一段隐藏的小程序代码,别的访客一打开页面,这段代码就在他们浏览器里偷偷运行——好比有人在公告栏贴了张纸条,凡读到的人钱包都被偷了。

价值 理解它才能防住:中招后访客的登录态(令牌)、cookie 会被窃取,黑客就能冒充他们登录。对有评论、发帖、私信功能的网站威胁极大。

何时用到 凡是用户能输入内容、又会展示给别人看的地方都有风险:评论区、昵称、帖子、聊天。做社区、留言、用户资料展示类功能时务必防范。

常见坑 根源是把用户输入原样塞进网页显示。正确做法是对用户内容做转义(把代码当普通文字显示)。前端框架默认会转义,但用 innerHTML、v-html 这类绕过手段时就破防了,要格外小心。

SQL注入

CSRF 跨站请求伪造

Token / JWT 令牌

CSRF 跨站请求伪造 CSRF (Cross-Site Request Forgery)

你登录了网银没退出,又点开一个钓鱼网站,它偷偷以你的名义向网银发了条'转账'指令。因为你浏览器还揣着网银的登录凭证,网银以为是你本人操作就执行了——好比有人借你刚签好字的空白支票去取钱。

价值 理解它才能防住:它专门冒用你已登录的身份做你没想做的事,如转账、改密码、删数据。对有重要操作的网站危害很大。

何时用到 凡是登录后能做敏感操作的网站都有风险:转账、改资料、下单、删除。做这类涉及状态变更的功能时需要防护。

常见坑 很多新手根本没听过它,默认就漏。防法是给关键操作加一个一次性校验令牌(CSRF token),坏网站拿不到。主流后端框架大多内置开关,记得开启,别关掉。

XSS 跨站脚本

Token / JWT 令牌

CORS 跨域

CORS 跨域 CORS (Cross-Origin Resource Sharing)

像小区门禁的访客白名单。浏览器默认不许 A 网站去偷偷读 B 网站的数据(防坏人),CORS 就是 B 网站贴出的告示:'我允许 A 网站来访问我'。名单上没有的,门卫(浏览器)直接拦下。

价 值 它是浏览器的一道安全机制,防止恶意网站偷读你在别处的数据。同时通过正确配置,让你自己的前端能合法地访问自己的后端接口。

何 时 用 到 做前后端分离时几乎必遇到。前端页面和后端接口部署在不同网址,前端调接口时浏览器就会用 CORS 检查,配错了请求就被拦,报红色错误。

常 见 坑 新手遇到 CORS 报错最常见的错误操作是图省事设成'允许所有网站'(*),等于把门禁全拆了。正确做法是只把你自己的前端域名加进白名单,别偷懒全放开。

CSRF 跨站请求伪造

API Key 密钥

认证

防火墙 Firewall Firewall

像小区大门的保安,站在你服务器最外层。规定哪些访客(网络流量)能进、走哪个门(端口)、从哪来,可疑的、没预约的一律挡在外面,只放行你明确允许的。

价 值 在攻击碰到你的程序之前就先挡一道,缩小被攻击的入口。能屏蔽恶意 IP、只开放必要的端口,是服务器安全的基础外层防护。

何 时 用 到 当你把网站部署到自己的服务器(而非托管平台)时需要配置。决定只开放网站用的端口、关掉数据库等不该对外的端口时,就在用防火墙。

常 见 坑 新手常把数据库端口、管理后台端口直接对全网开放,等于给攻击者留门。原则是只开必须开的,其余全关。另外别乱动远程连接(SSH)和网站端口,关错了自己也连不上。

速率限制

最小权限原则

CORS 跨域

最小权限原则 Least Privilege Least Privilege

给保洁阿姨配钥匙,只配她要打扫那几间房的,不给整栋楼的总钥匙。每个账号、每个程序只给它干活刚好够用的权限,多一分都不给,这样万一钥匙丢了,损失也限定在那几间房。

价 值 一种贯穿全局的安全思路:把每次出事的破坏范围压到最小。即使某个密钥泄露或某个账号被攻破,攻击者也只能动很有限的东西,不会一锅端。

何 时 用 到 配数据库账号、申请 API Key、设员工后台权限时都该想到。比如只读数据的程序就别给它删除权限,某个 Key 只用来发短信就别开它其他能力。

常 见 坑 新手图省事常给所有东西配最高权限(管理员、全功能 Key),一旦泄露就是灾难。正确做法是按需分配,宁可分多个小权限的账号,也别一个万能账号走天下。

授权

API Key 密钥

防火墙

第三方服务 · SaaS

06

不造轮子,花钱接现成服务,省时利器

15 词

软件即服务 SaaS (Software as a Service)

就像你不买一台洗衣机放家里,而是去自助洗衣店,投币就能用,机器坏了店家修,你只管洗衣服。软件也一样:不装在自己电脑上,打开网页登录账号就能用,坏了厂商修,你按月付钱。

价 值 省去自己开发和维护的巨大成本。一个功能(发邮件、收钱、存数据)别人已经做好了,你花钱直接接进来,几小时搞定原本要几个月才能自己造出来的东西。

何 时 用 到 用 AI 做一个网站或小程序时,几乎所有'非核心'功能都靠 SaaS:收款找 Stripe、发邮件找 Resend、登录找 Clerk。你只写自己最独特的那部分,其余全租现成的。

常 见 坑 按用量收费,用户一多账单会暴涨,上线前先算清楚价格曲线。还有'供应商锁定':深度依赖某家后想换会很痛,关键数据要能导出。

后端即服务

托管平台

支付网关

托管平台 Hosting (Vercel / Netlify)

你做好一道菜(写好的网站代码),但自家没餐厅没地址,客人没法来吃。托管平台就是帮你把菜摆上桌、开门营业、24 小时迎客的餐厅,还自带门牌号,谁都能找上门。

价 值 把你写的网站代码变成全世界打开浏览器就能访问的真实网址,不用自己买服务器、配环境。Vercel/Netlify 还能你一上传代码就自动更新线上版本。

何 时 用 到 用 AI 写完一个网页或前端应用,想让别人看到时第一步就是部署到托管平台。把代码连上 GitHub,几分钟就有一个能分享的网址,免费额度够个人项目用很久。

常 见 坑 免费额度对流量、运算时长有限制,项目火了或跑重任务会触发收费甚至限流。另外这类平台擅长前端,真正的数据库通常还得另配(如 Supabase)。

云厂商

域名注册商

后端即服务

后端即服务 BaaS (Supabase / Firebase)

开餐厅本来要自己建仓库、雇库管、装监控、办会员卡系统。BaaS 是一个'餐厅后厨全包'套餐:数据库、用户登录、文件存储、实时通知一次性配齐,你拎包入住,只管做菜(写界面)。

价 值 网站的'看不见的后半部分'(存数据、管用户、传图片)别人帮你搭好了,通过简单调用就能用。让一个不懂服务器的人也能做出有账号、能存数据的完整应用。

何 时 用 到 用 AI 做需要'用户注册登录 + 保存数据'的应用(待办清单、笔记、社区)时,接一个 BaaS 当后端最省事。Supabase 用的是标准数据库,日后想搬走也相对容易。

常 见 坑 省事的代价是把核心数据托管给别人,权限规则没配好可能导致数据被任何人读取(常见安全事故)。免费额度的数据库容量和连接数也有限。

软件即服务

认证服务

托管平台

支付网关 Payment Gateway

就像超市收银台的刷卡机。顾客的卡、银行、你的账户之间隔着复杂的流程,刷卡机(支付网关)是中间那个安全传话的人:核实卡有没有钱、把钱从顾客账户挪到你账户,你不用碰任何银行系统。

价 值 让你的网站能合法、安全地收钱。自己对接银行和处理信用卡极其复杂还涉及安全合规,支付网关把这一切打包,你接一下就能在网页上收款。

何 时 用 到 任何要在网站或小程序里卖东西、收会员费、做打赏时,都得接支付网关。它负责弹出付款页、处理刷卡、付款成功后通知你的程序发货。

常 见 坑 每笔交易要抽成(手续费),小额生意尤其要算。还有退款、争议扣款、跨境结算等规则,上线前看清条款,别等用户付了钱才发现到账有延迟或冻结。

Stripe 支付

支付宝/微信支付

软件即服务

Stripe 支付 Stripe

支付网关里最受程序员欢迎的那一家,像'收款界的乐高':文档清楚、积木式接入,几行代码就能在网页上摆出一个能刷卡的收银台,海外用户尤其认它。

价 值 国际收款的事实标准,接入简单、支持信用卡/订阅/分期等几乎所有场景。对做面向海外用户产品的 vibecoder 来说,Stripe 几乎是默认选择。

何 时 用 到 做一个面向国外用户的 SaaS、卖订阅、收美元时首选 Stripe。它有现成的'收银台页面'(Checkout),你几乎不用自己设计付款界面。

常 见 坑 美国标准费率约为每笔 2.9% + 0.30 美元(stripe.com/pricing),跨境卡还要加约 1.5%,小额交易那 0.30 美元占比很高。另外 Stripe 在中国大陆不能直接给本地商户收人民币,国内生意得用支付宝/微信。

支付网关

支付宝/微信支付

软件即服务

支付宝/微信支付 Alipay / WeChat Pay

在中国就像街边小店墙上那两张二维码。顾客扫一下、输密码,钱就到你账上。它们是国内做生意收钱的'默认收银台',几乎人人手机里都装着。

价 值 面向中国用户收款的必备渠道。中国人习惯扫码付款,不接这两个等于把绝大多数国内顾客挡在门外。Stripe 在国内收不了的钱,靠它们收。

何 时 用 到 做面向国内用户的小程序、公众号、网站要收钱时必接。微信小程序里收钱基本只能用微信支付;网页和 App 通常两个都接,让用户自选。

常 见 坑 开通要营业执照等资质,个人开发者门槛比 Stripe 高。接入文档比 Stripe 繁琐,签名、证书容易踩坑。费率通常约 0.6% 起(随行业和资质浮动),以官方商户后台为准。

支付网关

Stripe 支付

云厂商

认证服务 Auth Service (Clerk / Auth0)

就像办公楼大堂的门禁和前台:核对你是不是本人、发门禁卡、记录谁进了门。'谁是合法用户、密码对不对、能不能进'这套活儿,交给专门的保安公司,你不用自己造门禁。

价 值 用户的注册、登录、找回密码、微信/谷歌一键登录、双重验证等,自己写既繁琐又容易出安全漏洞。认证服务把这套打包,你接进来就有一个安全又好看的登录系统。

何 时 用 到 做任何需要'用户账号'的应用时,与其自己写登录(很容易写出漏洞),不如接 Clerk/Auth0。Clerk 还直接给现成的登录界面组件,几分钟就能用。

常 见 坑 按月活用户收费,用户多了账单涨得快。深度用了它的功能后想换很麻烦。安全核心交给第三方,要确认它的合规与可靠性。免费额度通常够早期用。

后端即服务

软件即服务

数据分析

邮件服务 Email Service (Resend / SendGrid)

你想给一万个客户寄信,自己一封封贴邮票寄不现实,而且'寄信人是无名小卒'的信容易被当垃圾扔掉。邮件服务就是专业邮局,帮你批量发、保证信能进对方收件箱而不是垃圾箱。

价 值 程序自动发邮件(注册验证码、找回密码、订单通知、营销邮件)如果用自己服务器发,十有八九进垃圾箱或被拉黑。专业邮件服务保证'送达率',让邮件真正到达用户。

何 时 用 到 做需要给用户发系统邮件的应用时接入,几乎是标配。注册要发验证码、下单要发确认信,都靠它。Resend 对程序员友好,接入很简单。

常 见 坑 免费额度有限,Resend 免费版约每月 3000 封、每天 100 封 (resend.com/pricing),超了要付费。发营销邮件还要遵守反垃圾邮件法规,域名也要做验证配置,否则照样进垃圾箱。

短信服务

软件即服务

后端即服务

短信服务 SMS (Twilio)

邮件服务的'短信版'。你想给用户手机发验证码或通知,自己没法直接连上全球运营商网络,短信服务就是那个帮你把短信发到任何人手机上的中间商。

价 值 手机验证码、登录确认、订单提醒这类要发到手机的短信,自己搞不定运营商对接, Twilio 这类服务一接就能全球发短信、打电话。

何 时 用 到 应用需要手机号验证码登录、或给用户发实时短信提醒时用。比如手机号注册要发 6 位验证码,就靠短信服务发出去。

常 见 坑 按条收费,且各国费率差别大,验证码被恶意刷会让账单爆炸(常见攻击),一定要加发送频率限制。国内发短信通常用阿里云/腾讯云的短信服务更便宜合规。

邮件服务

认证服务

云厂商

数据分析 Analytics (GA / PostHog)

就像商店门口装的客流计数器加监控:多少人进店、看了哪些货架、在哪儿转身就走、最后谁结账了。网站装上分析工具,就能看清访客都在你的页面上干了什么。

价 值 不靠猜,而是用真实数据知道有多少人来、从哪来、点了什么、在哪一步流失。这是优化产品、判断哪个功能有用的眼睛。

何 时 用 到 网站或应用上线后想知道'到底有没有人用、用得怎么样'时接入。GA(谷歌分析)是经典免费选择;PostHog 更适合产品功能层面的细致追踪。

常 见 坑 埋点(记录用户行为的代码)装太多会拖慢网页、引发隐私合规问题(很多地区要 cookie 同意弹窗)。数据采了不看等于白采,先想清楚要回答什么问题再装。

软件即服务

认证服务

托管平台

域名注册商 Domain Registrar

网址(如 `mysite.com`)就像店铺的门牌号和店名,不能凭空拥有,得去'工商局'登记并按年续费。域名注册商就是这个登记处,你交钱注册一个名字,这名字这一年就归你用。

价 值 让你的网站有一个好记的网址,而不是一串数字 IP。注册商负责把这个名字登记到你名下,并把它指向你网站所在的服务器。

何 时 用 到 网站做好部署后,想要一个像样的专属网址而不是平台给的随机地址时,去注册商买个域名。常见有 Namecheap、Cloudflare、阿里云万网等。

常 见 坑 首年常有低价促销,续费价可能贵几倍,买前看清续费价。别忘了续费,过期会被别人抢注。隐私保护(WHOIS)有的免费有的收费,留意别泄露个人信息。

托管平台

云厂商

软件即服务

云厂商 Cloud Provider (AWS / 阿里云)

像一家超大型'算力商场':服务器、数据库、存储、AI 算力等几百种东西按需租用,用多少付多少。前面那些专门服务(托管、BaaS)很多其实就建在这些云厂商之上。

价 值 提供互联网最底层的基础设施。需要更大、更灵活、更专业的能力时(海量存储、自定义服务器、大模型算力),云厂商什么都能租,是整个行业的地基。

何 时 用 到 vibecoder 早期通常用不到原始云厂商,因为太复杂;一般先用 Vercel、Supabase 这类'封装好的'。当项目规模变大或有特殊需求时,才直接碰 AWS/阿里云。

常 见 坑 功能极多、配置极复杂,新手容易开了忘关导致莫名扣费(著名的'天价账单'故事)。一定要设预算告警。国内用阿里云/腾讯云,海外用 AWS,选错地区会很慢。

托管平台

后端即服务

域名注册商

API 中转/聚合平台 API Aggregator / Proxy

像一个'万能遥控器':不用为每个电器单独配遥控器,一个就能控所有牌子的电视空调。它把好几家不同公司的接口,统一成一个入口,你只对接一次就能调用多家服务。

价 值 对接 AI 大模型尤其有用:不用分别在 OpenAI、Anthropic、谷歌各开账号充值,一个聚合平台一个 key 就能调所有模型,还能自动切换更便宜或更快的那家。

何 时 用 到 用 AI 编程做需要调用大模型的应用时常用。想同时试好几个模型、或国内访问海外模型不方便时,中转平台提供一个统一稳定的入口(如 OpenRouter)。

常 见 坑 多了一层中间商,可能加价、增加延迟、甚至看到你传的数据,敏感内容慎用。便宜的小中转平台可能跑路或偷偷换成劣质模型,选有口碑的。

云厂商

软件即服务

支付网关

内容管理系统 CMS (Content Management System)

像公众号的后台编辑器:不懂代码的人也能写文章、传图、发布,前台网站自动更新。CMS 就是网站的'后台编辑室',让运营人员不碰代码就能改内容。

价 值 把'网站长什么样'和'网站写什么内容'分开。程序员搭好框架后,写文章、改产品介绍、换 banner 这类日常更新,交给非技术同事在后台点点鼠标就能完成。

何 时 用 到 做博客、官网、电商这类内容经常更新的网站时用。WordPress 是经典老牌;现在 vibecoder 常用 Sanity、Strapi 这类'无头 CMS',只管内容、前端自己用代码做。

常 见 坑 WordPress 插件多但安全漏洞也多,要勤更新。'无头 CMS'灵活但要自己写前端对接。别为一个几乎不更新的页面套个重型 CMS,反而把简单事复杂化。

低代码/无代码

托管平台

软件即服务

低代码/无代码 No-code / Low-code

像用积木或拼图搭模型,不用一砖一瓦地砌。在网页上拖拖拽拽、填填表单,就能拼出一个能用的网站或小工具,几乎不用写代码。

价 值 让完全不懂编程的人也能做出真实可用的产品(网站、表单、自动化流程、简单 App)。对验证想法、做内部工具、抢时间上线极其高效。

何 时 用 到 想快速验证一个点子、做个简单的落地页或表单、或搭内部管理工具又不想写代码时用。常见有 Webflow、Bubble、飞书多维表格、Zapier 等。

常 见 坑 简单需求很快,但需求一复杂就撞天花板,定制能力有限。平台一涨价或关停你就被动,数据和逻辑都困在里面。复杂产品最终往往还得回到写代码。

内容管理系统

托管平台

软件即服务

补充高频词

07

审稿人补漏的高频遗漏概念

12 词

Vibe Coding 氛围编程 Vibe Coding

以前盖房子要自己砌砖,现在你只用嘴跟一个超级靠谱的施工队说"我想要个带泳池的两层小楼",它就边问边盖。你描述想法,AI 写代码,你看效果不对再让它改。

价 值 让完全不懂代码的人也能做出能跑的网站、小程序、工具。你的核心工作从"会不会写"变成"会不会清楚地描述需求、会不会判断做出来的东西对不对",门槛从语法降到了表达。

何 时 用 到 这就是你现在干的事本身。用 Cursor、Lovable、Claude 这类工具,靠对话一步步把脑子里的产品做出来,就是 vibe coding。它在 2025 年被柯林斯词典选为年度词汇,2026 年已是主流做法。

常 见 坑 AI 能跑通不等于能上线。生成的代码常有安全漏洞、性能坑、改一处崩一片。别盲信"它说能用",关键功能(尤其涉及钱和用户数据)一定要测、要让 AI 自查安全,复杂项目最好找懂行的人过一遍。

人工智能

代码编辑器 /

提示词

Bug

AI 编程工具 / AI 代码编辑器 AI Code Editor (Cursor, Lovable, Bolt, v0)

普通文本编辑器像一张白纸加一支笔,AI 编程工具则像一个坐在你旁边、能看懂你整个项目、随叫随到帮你写帮你改的高级助手,你说一句它动手一片。

价 值 把"自己一行行敲代码"变成"对话式开发"。不同工具分工不同:Cursor 适合已经有代码、想认真维护的项目;Lovable、Bolt、v0 适合从零快速搭原型、做界面,几句话就出一个能看的页面。

何 时 用 到 你做 vibe coding 的主战场。想快速验证一个点子、做个落地页或小程序原型,用 Lovable/Bolt;项目变复杂、要长期维护,常见做法是"先在 Lovable 搭原型,再转到 Cursor 精修"(业内叫毕业工作流)。

常 见 坑 原型工具(Bolt/v0)做 demo 飞快,但做出的东西往往难维护,真要上线常得重写或迁移。别把"演示能跑"当成"产品做完了",选工具要看你是要快还是要长久。

Vibe Coding 氛围编程

代码编辑器 /

大语言模型

技术选型

HTML / CSS / JavaScript HTML / CSS / JavaScript

做一个网页就像装修房间:HTML 是毛坯结构(哪里放标题、哪里放图片、哪里放按钮),CSS 是装修风格(颜色、字体、间距、排版好不好看),JavaScript 是让房子会动的电路(点按钮弹窗、提交表单、加载数据)。

价 值 这是所有网页的三块地基,浏览器只认这三样。哪怕你全程让 AI 写,听懂这三个词能让你和 AI 沟通时一句话点到位:"这是 CSS 的问题,帮我把按钮调大"比"那个东西不好看"高效十倍。

何时用到 你做任何网站、小程序、落地页时,AI 生成的就是这些。当你想说"布局乱了"(HTML)、"配色丑"(CSS)、"点了没反应"(JavaScript),知道是哪一块出问题,描述需求会准得多。

常见坑 新手容易把三者混为一谈,跟 AI 说"代码不对"却不说是结构、样式还是交互,导致来回返工。另外别一看 JavaScript 就和后端混淆——它主要跑在浏览器(前端)里。

前端

框架

开发者工具 / F12 控制台 DevTools / Browser Console

网页出问题时,F12 就像汽车的引擎盖,掀开能看到里面的仪表和报错灯。普通用户只看到"页面白了",掀开盖子你能看到具体哪根线断了、报了什么错。

价 值 排错的第一现场。页面打不开、按钮没反应、图片不显示时,按 F12 打开控制台(Console),里面那行红字就是错误原因。把这行红字原样复制给 AI,它十有八九能直接告诉你怎么改。

何时用到 几乎每次 vibe coding 调试都用得上。AI 生成的网页在浏览器里跑出 bug,与其干瞪眼描述"它坏了",不如 F12 截图或复制报错喂给 AI,排错效率天差地别。

常见坑 很多新手不知道这个工具存在,遇到问题只会反复说"不行",却没把浏览器里现成的报错信息交给 AI。报错原文(尤其红色那段)是最有价值的线索,千万别忽略。

调试

Bug

日志

本地运行 / localhost Localhost (localhost / 127.0.0.1)

localhost 就是你自己电脑上的"内部试营业"。东西做好了先在自家厨房尝尝味道(只有你能看到),觉得 OK 了再开门迎客(部署上线让全世界访问)。

价 值 让你在花钱上线之前,先在自己电脑上免费、快速地预览和测试。改一行看一眼效果,不用每次都传到服务器,迭代飞快。地址通常长这样:http://localhost:3000。

何 时 用 到 AI 帮你跑起项目后,常让你打开 localhost:某个端口 在浏览器里看效果。这是"还没上线、只有你能看"的阶段,确认没问题了才走部署上线。

常 见 坑 最常见的困惑:localhost 上好好的,发给朋友打不开——因为它只存在于你这台电脑,别人访问不了。想让别人看,必须先部署到服务器或托管平台。

部署

开发/测试/生产环境

端口

上线/生产环境

构建 / 编译 Build Build / Compile

你写的代码像一份手写菜谱(方便人读),但服务器和浏览器吃的是"预制半成品"。构建就是把菜谱批量加工、打包、压缩成机器能高效运行的成品的过程。

价 值 上线前的必经一步。开发时为了好读好改,代码是松散的;构建会把它压缩、合并、优化成又小又快的版本。很多部署平台会在你推送代码后自动帮你构建。

何 时 用 到 部署网站时几乎必然遇到。常见场景是"build failed(构建失败)"——AI 写的代码本地看着对,一构建就报错。把构建报错日志整段喂给 AI,通常能定位到具体哪行有问题。

常 见 坑 新手常被"本地能跑但构建失败"搞懵。原因往往是少装了依赖、或代码有本地容忍但构建严格的小错。别慌,构建日志里的报错信息就是答案,完整复制给 AI。

部署

CI/CD 持续集成与部署

包/依赖

Bug

响应式设计 Responsive Design

同一杯水,倒进高脚杯、马克杯、矿泉水瓶都要能装、都要好看。响应式设计就是让同一个网页,在手机、平板、电脑上自动调整排版,屏幕大就铺开、屏幕小就堆叠。

价 值 决定你的网站在手机上能不能用。现在大多数访客用手机,如果只在电脑上看着好、手机上字小到要捏放大,直接劝退用户。做对了,一套页面通吃所有设备。

何 时 用 到 做任何对外的网站、落地页、小程序时都要确认。AI 默认生成的页面不一定手机友好,你要主动让它"做成响应式,在手机上也好看",并且自己拿手机或 F12 切到手机视图实测。

常 见 坑 只在电脑大屏上验收,上线后才发现手机上排版崩了。一定要主动测手机端——浏览器 F12 里有切换设备尺寸的功能,或直接用手机打开链接看。

前端

HTML / CSS / JavaScript

开发者工具 / F12 控制台

数据库迁移 / Migration Database Migration

数据库像一个已经住满人的档案柜,迁移就是"在不丢现有档案的前提下改造柜子结构"——比如新加一个抽屉(加字段)、给抽屉改名、或把档案重新归类,且每一步都有记录可回退。

价 值 让你安全地修改已经装着真实数据的数据库结构。直接手动改容易丢数据、改错回不去;迁移把每次结构变更变成可执行、可追溯、可撤销的步骤,团队和服务器之间也能保持结构一致。

何 时 用 到 产品上线后想加新功能,常要给数据库加新表或新字段(比如用户表加"会员等级")。这时 AI 会帮你生成迁移脚本。这属于高风险操作,动线上真实数据前务必先备份。

常 见 坑 在有真实用户数据的库上直接乱改字段是大忌,可能永久丢数据且无法恢复。改结构前必须先备份;迁移脚本要先在测试环境跑通,确认没问题再上生产。

数据库

ORM 对象关系映射

备份

开发/测试/生产环境

备份 Backup Backup

备份就是给你的重要东西拍个"存档照片"另存一份。手机掉了不心疼,因为照片云端有备份;同理,服务器崩了、数据库删错了,只要有备份就能还原,没备份就是灾难。

价值 你最后的救命稻草。代码可以重写,但用户数据(订单、注册信息、上传内容)丢了往往无法重来。定期备份数据库和关键文件,意味着"最坏情况下也能恢复到昨天"。

何时用到 任何动数据库结构、批量删数据、做重大升级之前,第一件事就是备份。日常也应让数据库自动定时备份。很多云数据库和托管平台自带自动备份功能,开启它。

常见坑 最惨的不是没备份,是"以为有备份"——从没验证过能不能恢复。备份要定期真的还原一次试试,确认它有效;同时别把备份和原数据放在同一台服务器,一起没了就白搭。

数据库

数据库迁移 /

云服务器/

对象存储

定时任务 / 计划任务 Cron Cron / Scheduled Job

定时任务就是给服务器设的"自动闹钟"。你不用每天守着手动点,设好"每天凌晨 3 点自动备份数据库""每周一早上 9 点群发邮件",到点它自己执行,你睡觉它干活。

价值 把重复性、按时间触发的活儿交给机器自动跑。定时备份、定时清理过期数据、定时拉取数据更新、定时发提醒——凡是"每隔多久要做一次"的事,都靠它,省人力还不会忘。

何时用到 产品做大后常需要。比如想做"每天早上自动给用户推送昨日数据",或"每小时同步一次外部价格",就让 AI 帮你配一个定时任务(在服务器上叫 cron,在托管平台上常叫 Scheduled Functions)。

常见坑 定时任务出错往往很隐蔽——它在后台静默运行,失败了你可能几天都不知道。一定要加日志和失败通知,别让一个挂掉的备份任务静悄悄欠你一周的数据。

服务器

Serverless 无服务器

日志

备份

爬虫 / 数据抓取 Web Scraping / Crawler

爬虫就像派一个不知疲倦的助理,按你的指令去一个个网页上"抄数据"——把商品价格、文章列表、评论等批量复制下来整理成表格,人工抄一天的活它几分钟搞定。

价 值 当你需要的数据散落在别人的网页上、又没有现成接口给你时,爬虫帮你自动批量收集。做比价、行业调研、内容聚合、训练数据收集时常用,把网页上的信息变成你能用的结构化数据。

何 时 用 到 你想做个"自动汇总某网站最新信息"的工具,但对方没提供 API,就会让 AI 帮你写爬虫去抓。注意优先找官方 API,有 API 别爬,爬虫是没有正规接口时的退路。

常 见 坑 两大坑:一是法律和合规,抓取受版权保护或个人隐私数据可能违法,很多网站的使用条款也禁止爬取,商用前务必确认合规;二是抓太猛会把人家服务器搞崩、把自己 IP 封掉,要控制频率。

API 应用程序接口

速率限制

IP地址

数据分析

开源协议 / 许可证 Open Source License (MIT, GPL, Apache)

开源代码不等于"随便拿随使用"。许可证就像借东西时贴的便利贴:有的写"随使用,标个我名字就行"(MIT),有的写"你用了我的,你做的也必须开源"(GPL),规矩差别很大。

价 值 决定你能不能、以及怎样合法地把别人的开源代码用进你的商业项目。选对了省时省钱,选错了可能被迫公开自己的全部代码,甚至吃官司。看一眼协议类型,几秒钟避开大坑。

何 时 用 到 你用 AI 搭项目时,会大量引入开源库和模板。商用产品尤其要留意:MIT、Apache 这类"宽松协议"基本随使用;GPL 这类"传染性协议"可能要求你的项目也开源,商用前要看清。

常 见 坑 最危险的是"拿来就用、从不看协议"。GPL 协议的代码进了你的闭源商业产品,理论上你得把整个产品代码也公开。不确定时,直接问 AI"这个协议能用于闭源商业项目吗"。

开源

库

GitHub / 代码仓库

技术选型

怎么用这本词典 · 遇到看不懂的词先搜一下;每个词条底部的灰色标签是**关联词**,点一下直接跳过去,顺藤摸瓜建立全貌。

内容由多智能体并发调研 + 人工校订生成,面向零基础 vibecoder,力求准确好懂;价格/服务等时效信息以官方为准。

© 2026 张拼拼 (Max Pin) · zpplife@gmail.com · 保留所有权利

导出说明：原网页含 1 处已损坏的字符，本文件以“【原文缺字】”标明，未推测或改写缺失文字。